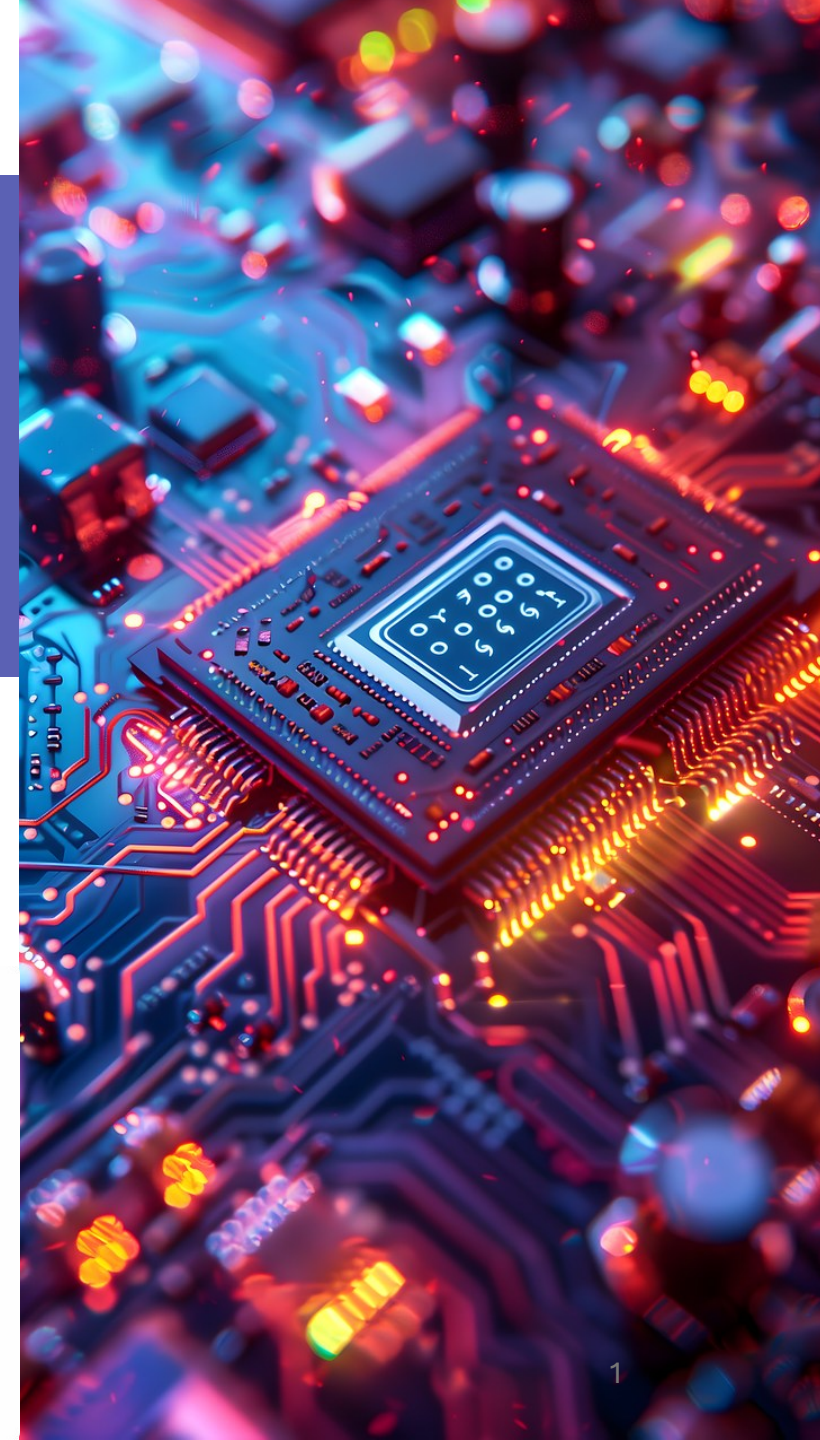


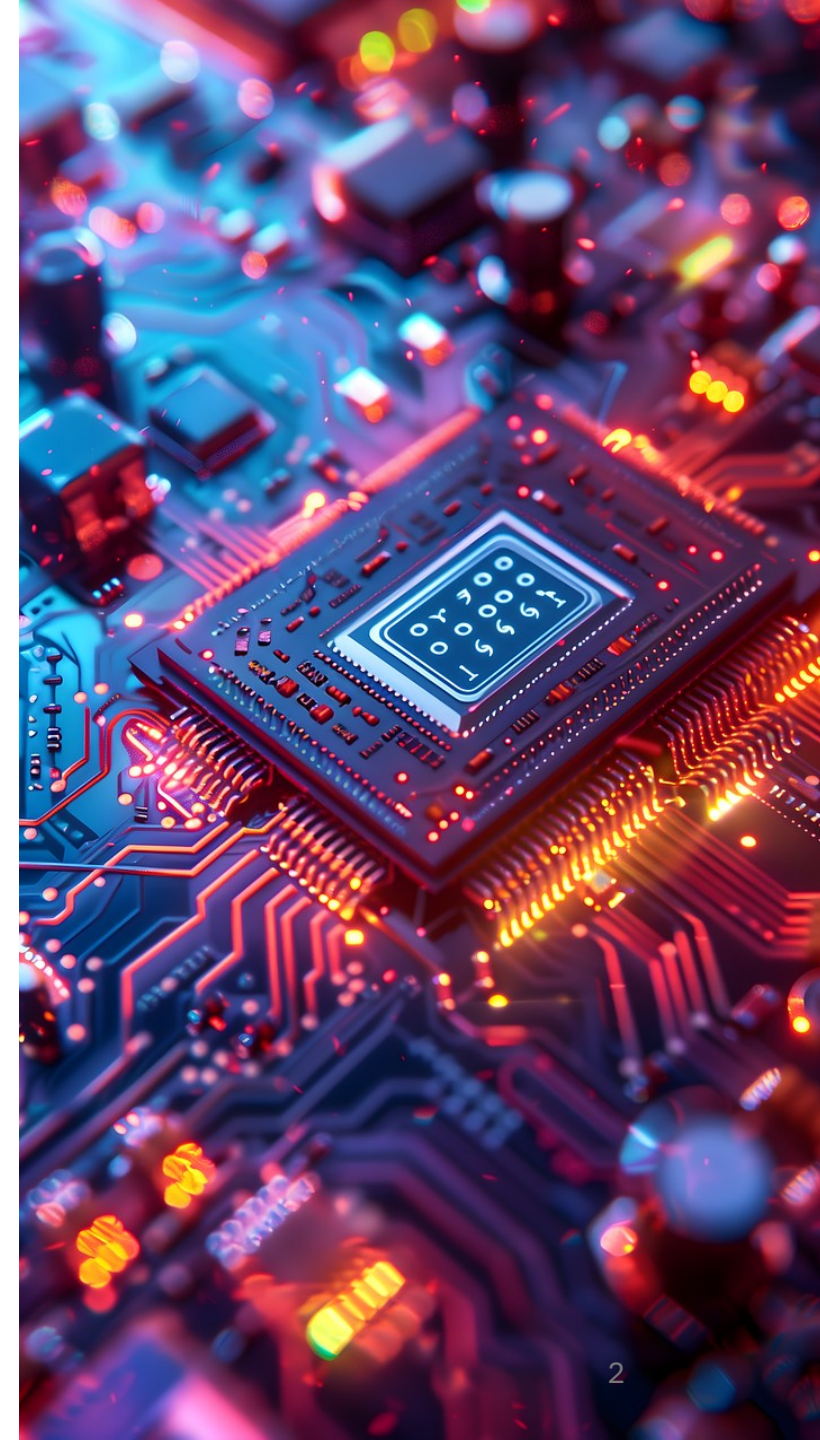
Crea un Circuito Integrado con Software Libre

Agosto 27 de 2026



Agenda

- Orígenes y perspectivas de la industria de IC
- El flujo de diseño
- Tecnología CMOS
- Nuestro primer chip digital en Wokwi
- Flujo de diseño *semicustom* en HDL + OpenLane
- Envío de archivos a TinyTapeout



¿Qué es IEEE CASS?

<https://ieee-cas.org/>

- **Sociedad de Circuitos y Sistemas del IEEE**
- **Misión:**
 - Fomentar innovación tecnológica
 - Tecnologías emergentes
 - Aplicaciones circuitos y sistemas en beneficio de la humanidad
- **Áreas Estratégicas de CASS**
 - **Nuevos paradigmas** en ing. circuitos y sistemas: interfaz, computación y conectividad
 - Aplicaciones de circuitos y sistemas incluyendo:
 - alimentación, agricultura, atención sanitaria y ciencias de la vida, tecnología centrada en el humano, movilidad y ciudades inteligentes, y energía sostenible
 - Teoría fundamental que sustenta tecnologías emergentes



Oportunidades en IEEE CASS

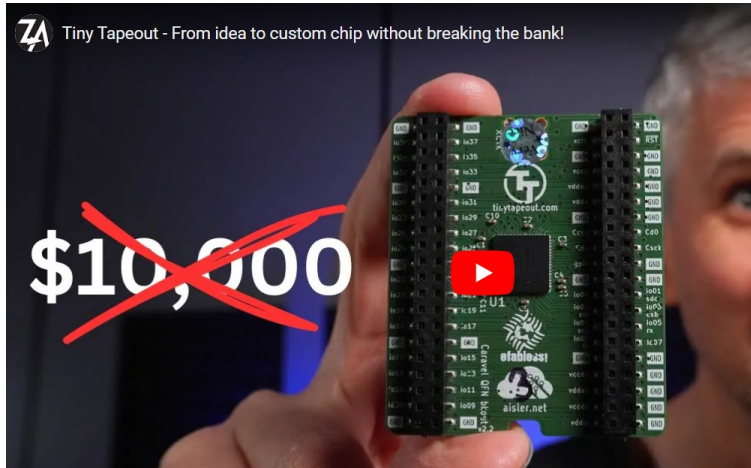
<https://ieee-cas.org/>



- Descuentos especiales en conferencias y workshops CASS
- Becas especiales y oportunidades
 - Seasonal Schools
 - Pre-Doctoral Scholarship
- Student Travel Awards (4 llamados al año)
- Iniciativas Young Professionals
- Capítulos estudiantiles:
 - Apoyos Student Branch Chapter Activity Outreach (Hasta 1000 USD)

Webinars semanales
(Viernes 11:30am en YouTube)
www.youtube.com/@cassriograndedosul

Iniciativas de Universalización de CI



<https://tinytapeout.com/>



<https://ieee-cas.org/universalization-ic-design-cass-unic-cass>

Tutoriales <https://unic-cass.github.io/>



IEEE DIVISION 1

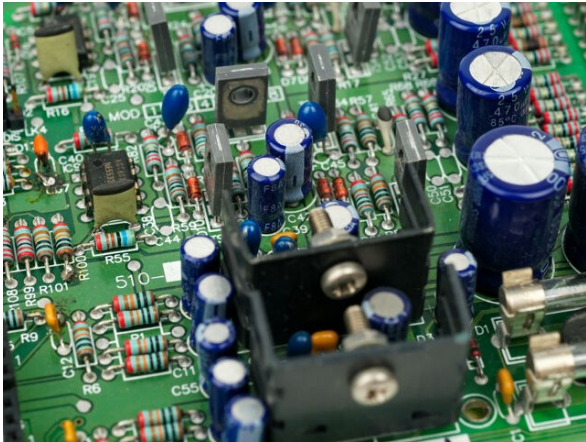
Microelectronic bootcamps



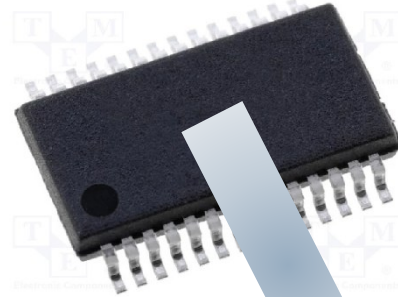
Orígenes y perspectivas de la industria de IC

¿Qué es un Circuito Integrado?

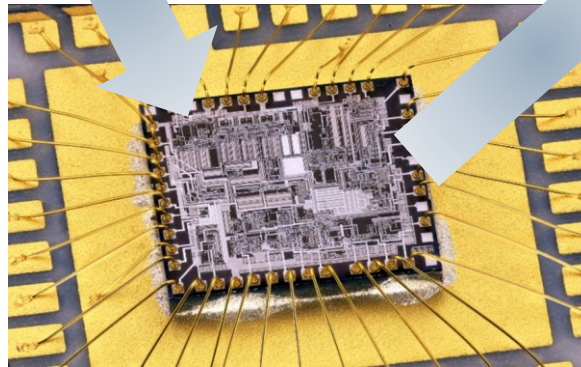
Circuito Discreto



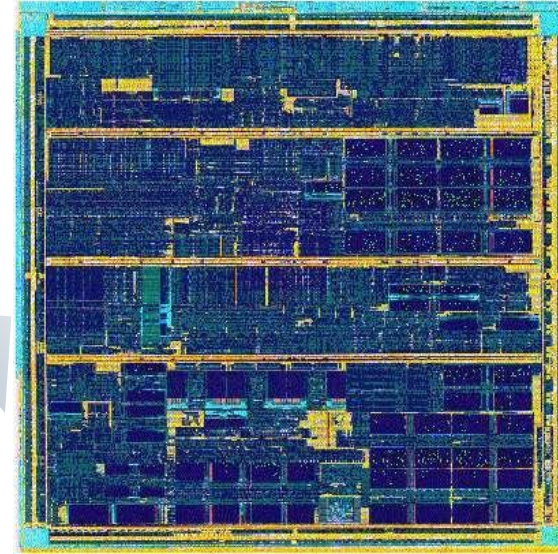
Integrated Circuit (IC)



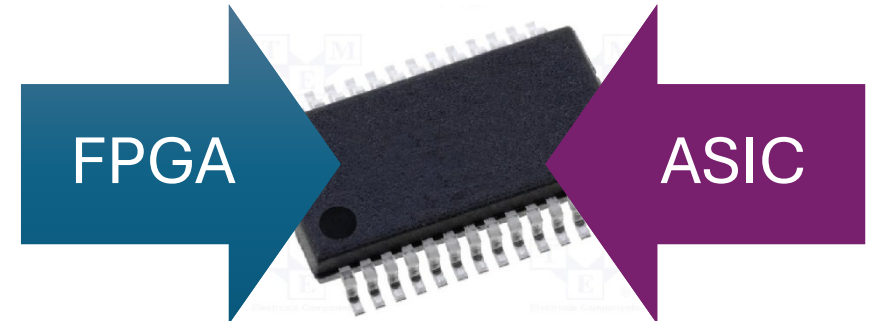
Packing



Wire bonding

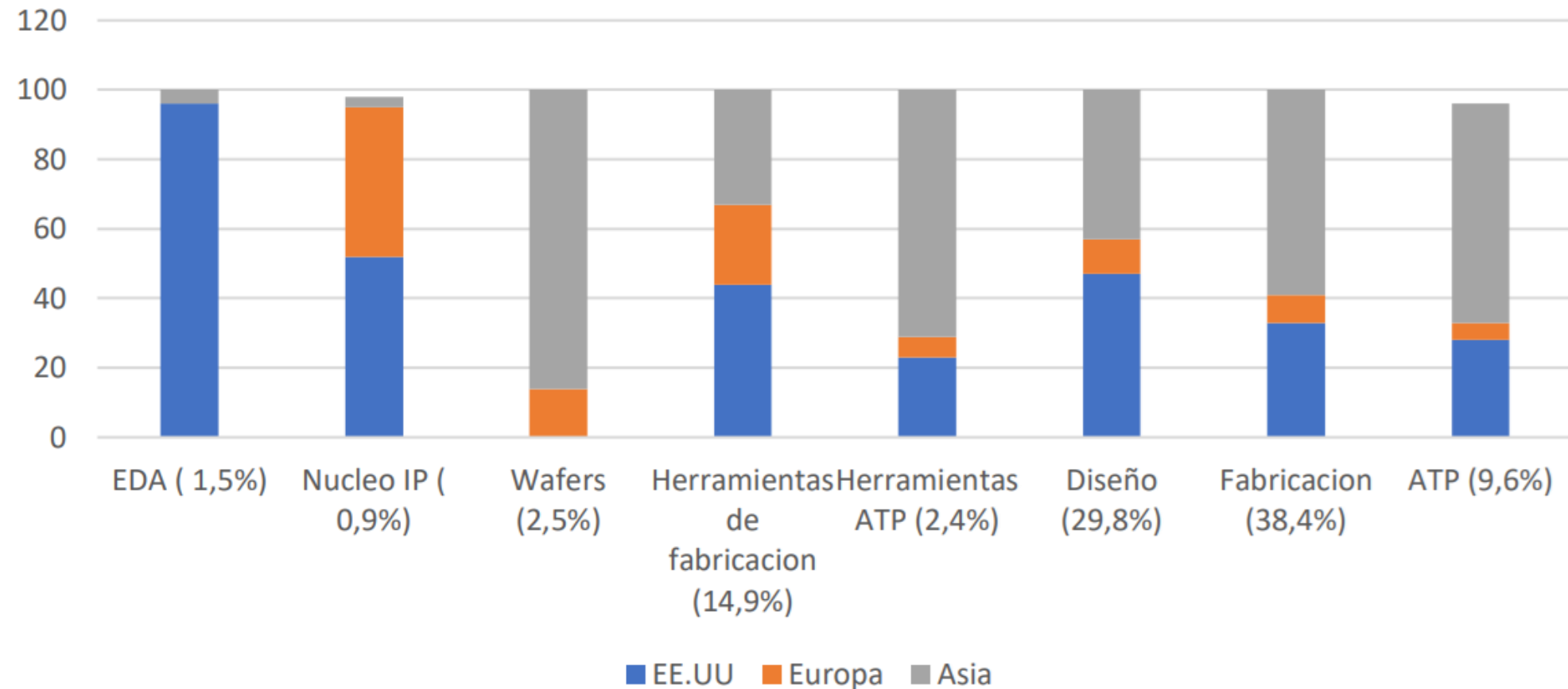


Die



Cadena de valor de la industria de IC

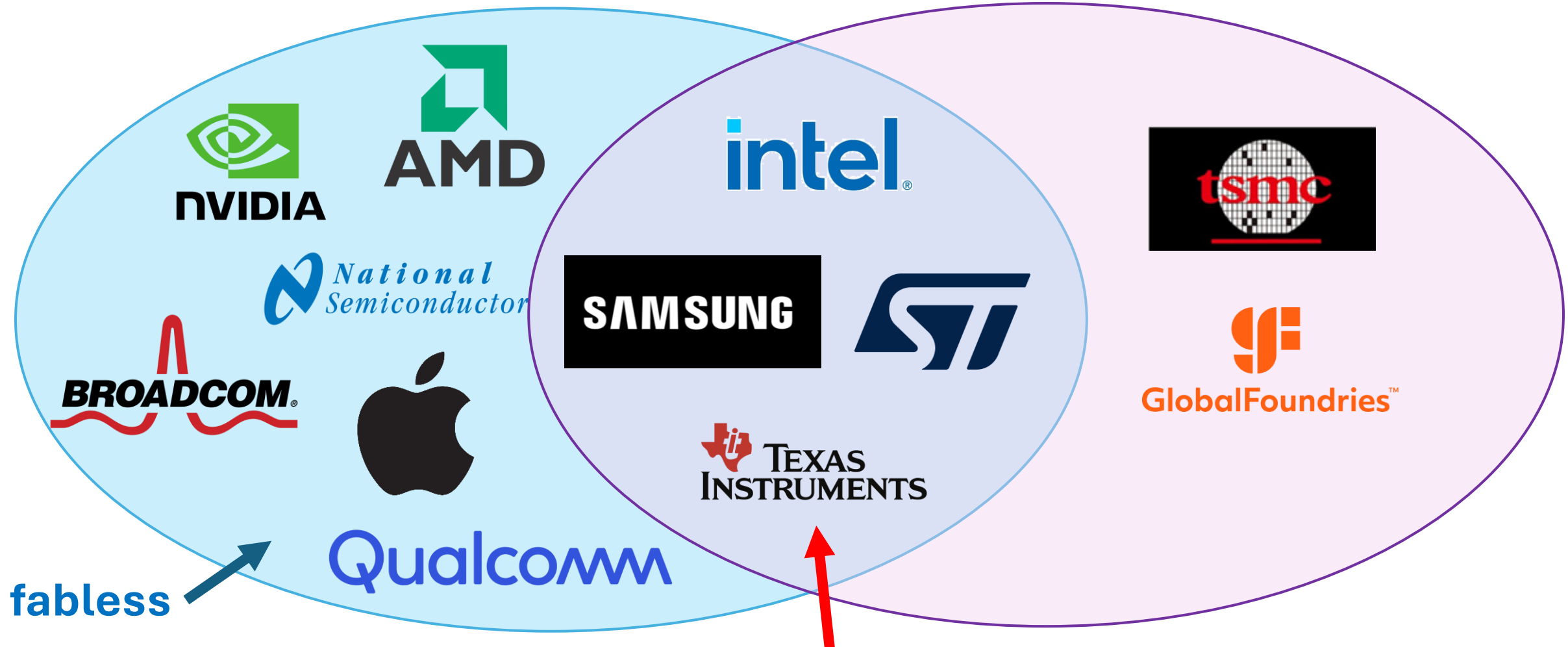
Distribución geográfica de los principales componente de la cadena de producción
(en parentesis peso relativo de los costos)



Cadena de valor de la industria de IC

Design Houses

Manufacturing Fabs



IDM (Integrated Device Manufacturer)

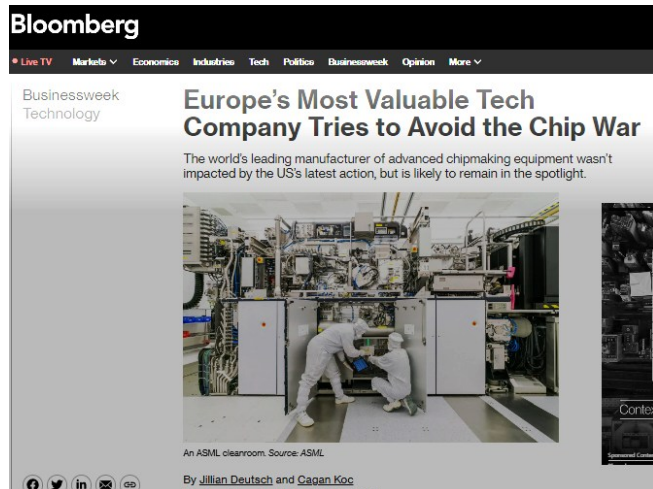
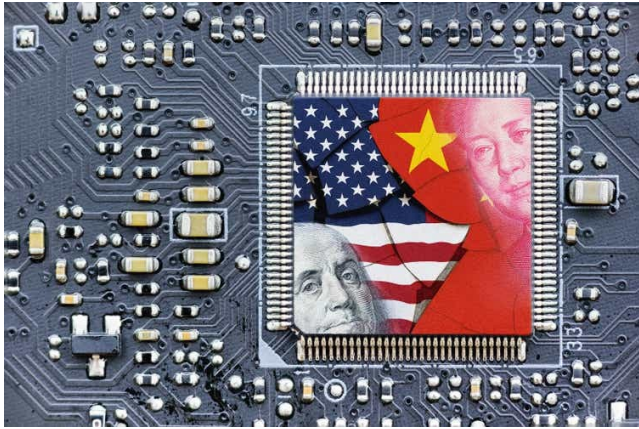
Cadena de valor de la industria de IC

¿Cómo estamos en Latinoamérica?



¿Necesitamos formar ingenieros en Colombia que sepan diseñar IC?

Guerra de los chips

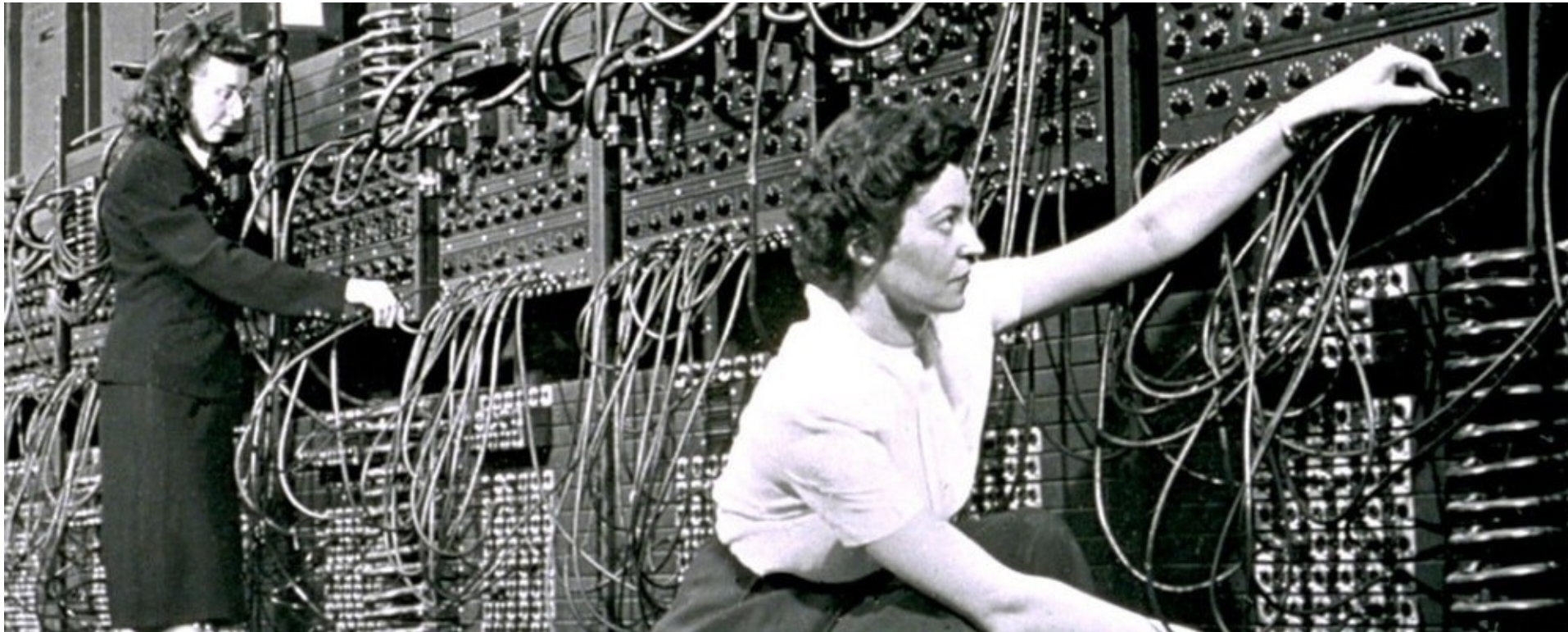


Carencia global de ingenieros electrónicos



Proyecto de Ley 047 de 2024:
Por el cual se fomenta la industria
electrónica y de semiconductores
en Colombia

¿Cómo llegamos aquí?



1944 - ENIAC (Primer computador electrónico)



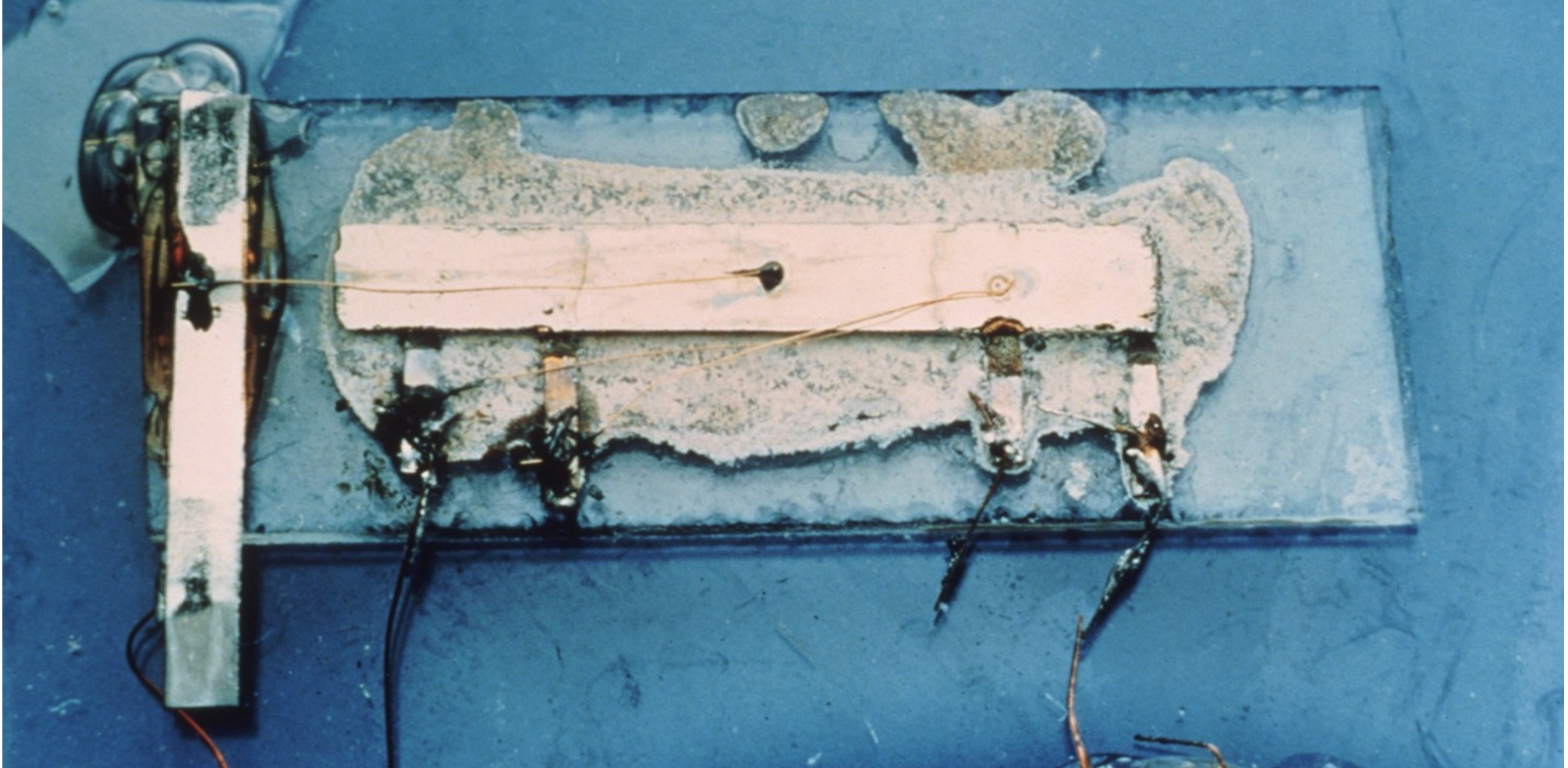
¿Cómo llegamos aquí?



1947 – Primer Transistor

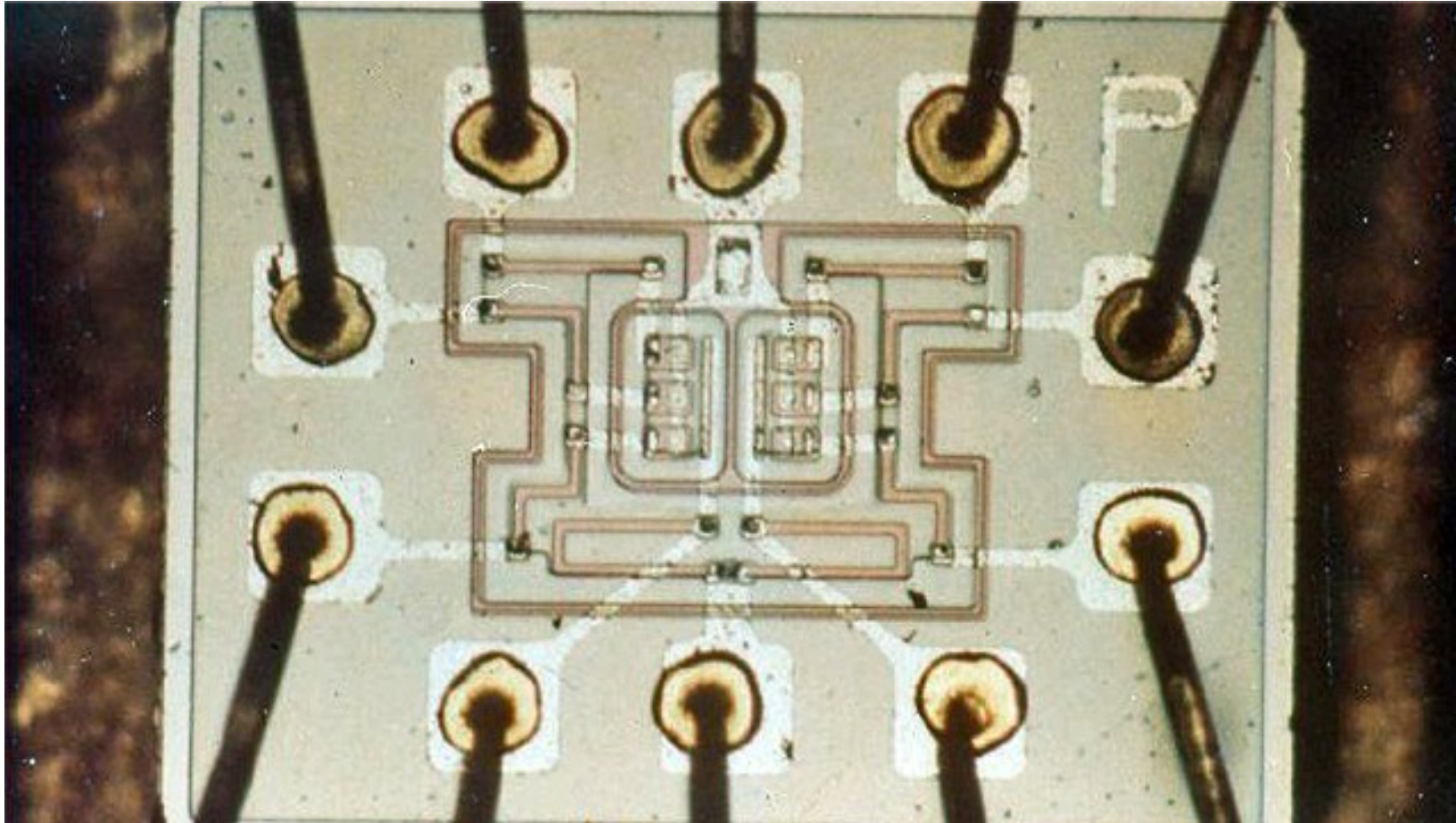


¿Cómo llegamos aquí?



1958 – Primer Circuito Integrado

¿Cómo llegamos aquí?

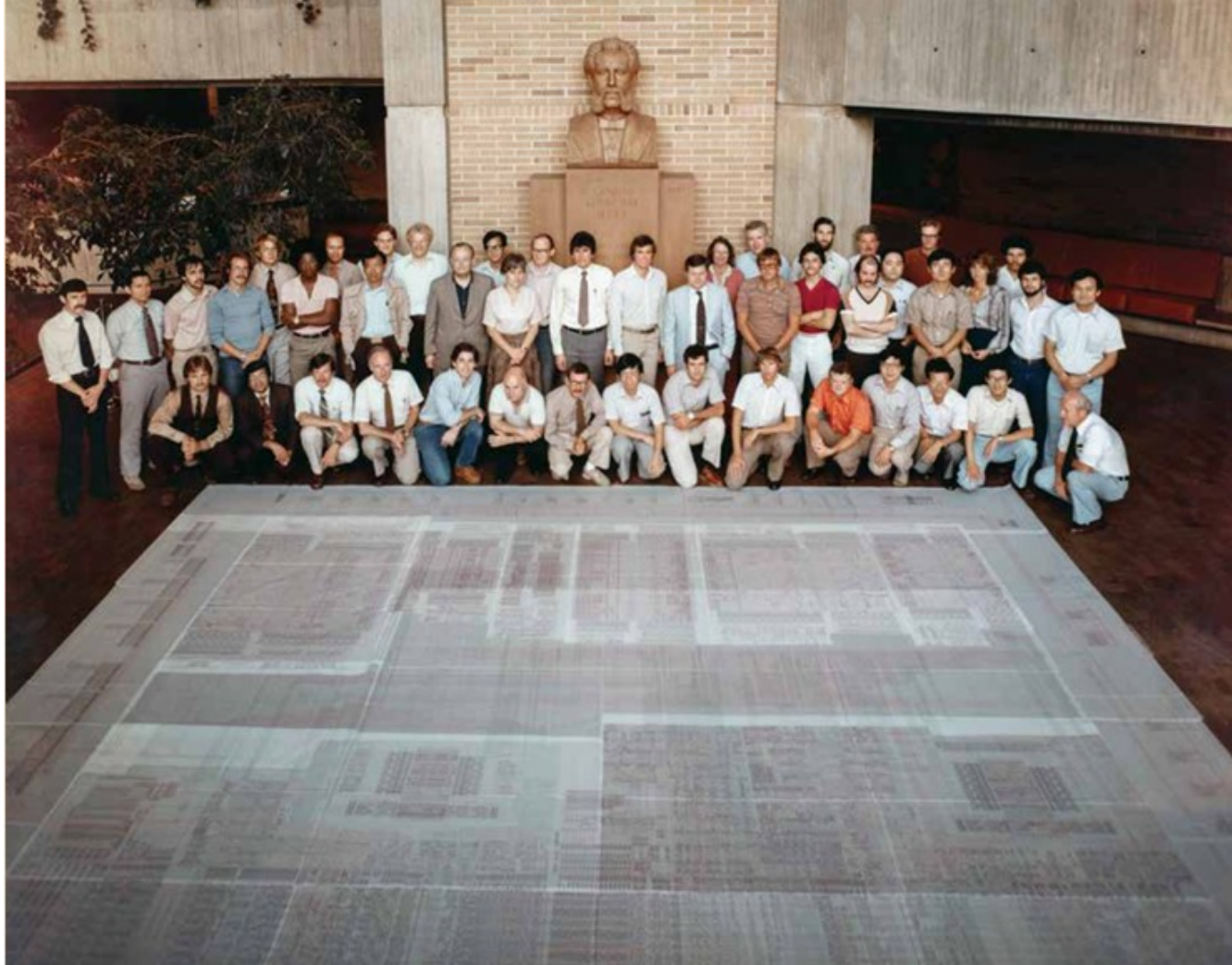


Compuerta NOR – Computadora Misiones Apollo (1960's)

¿Cómo llegamos aquí?

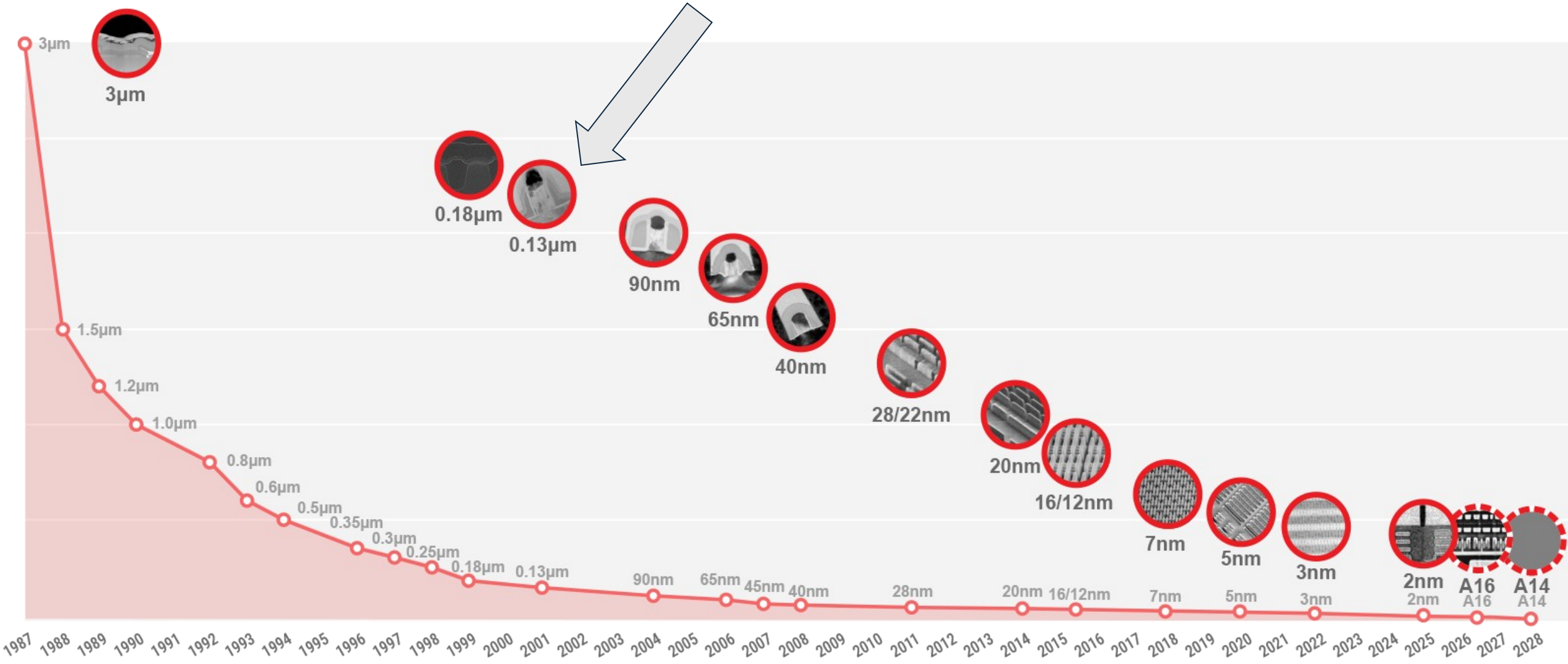


¿Cómo llegamos aquí?



Microprocesador BELLMAC 32bit (1980) – Tecnología 2,5 μ m

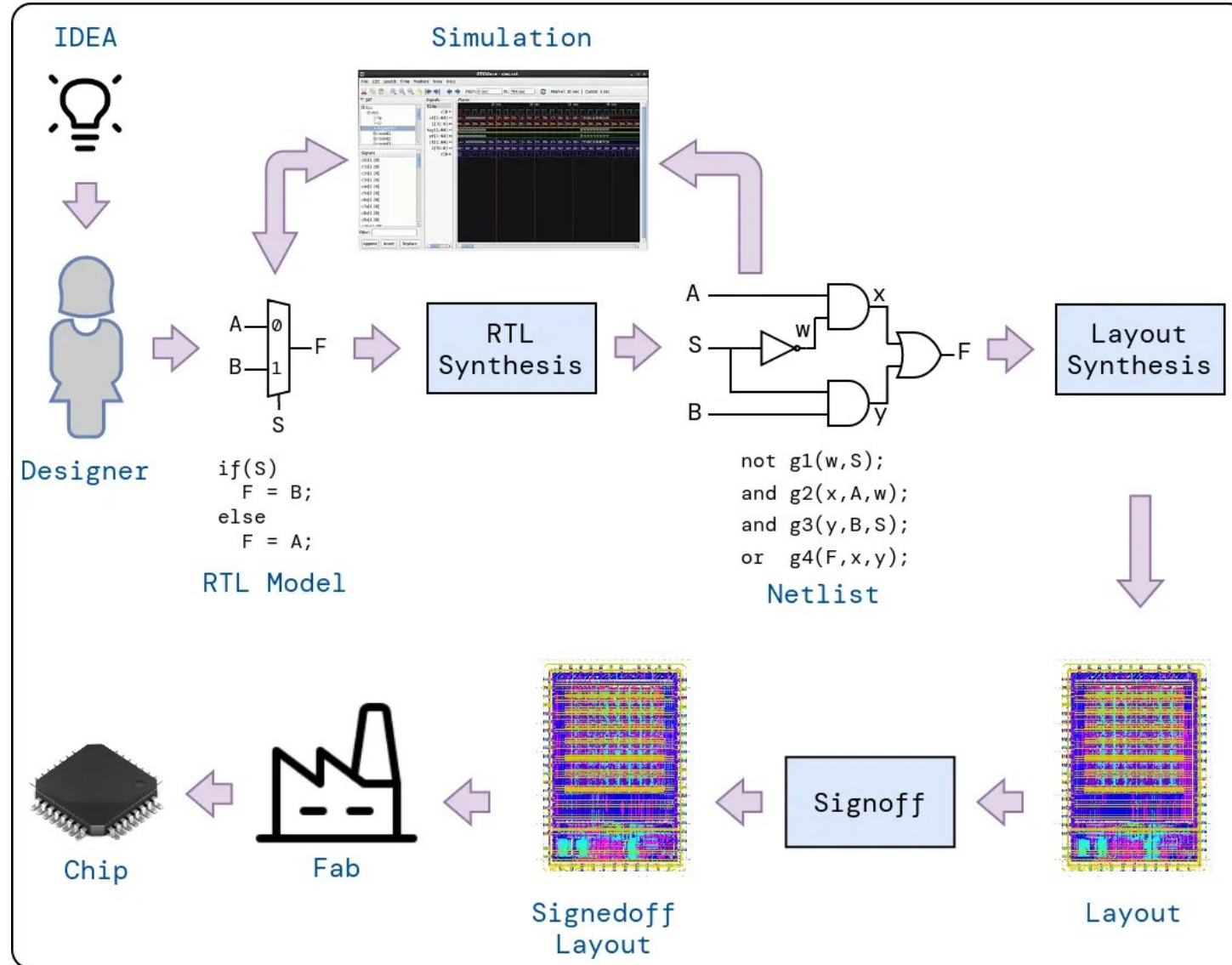
¿Cómo llegamos aquí?



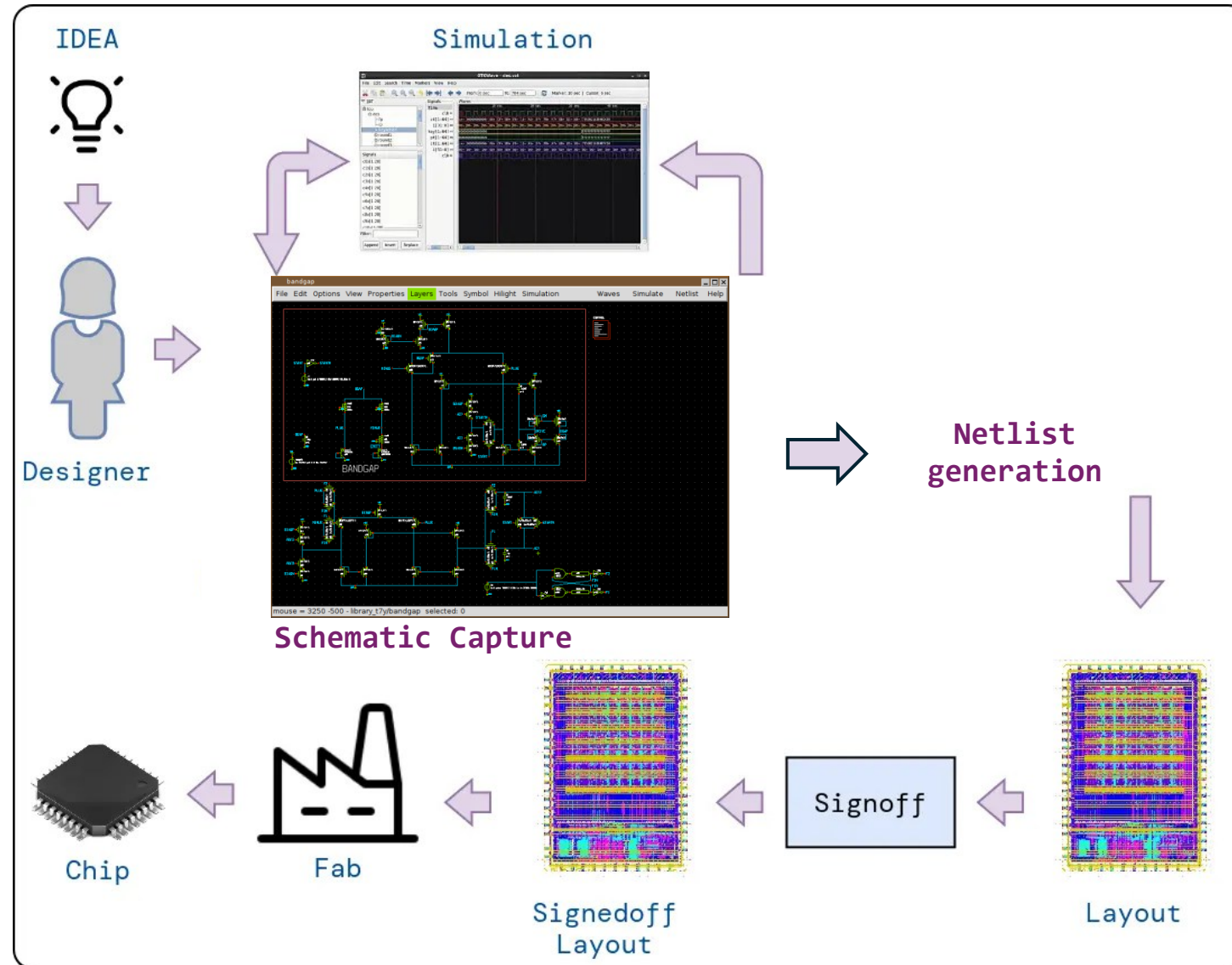
https://www.tsmc.com/english/dedicatedFoundry/technology/logic/l_A16

El flujo de diseño

Flujo de Diseño Digital

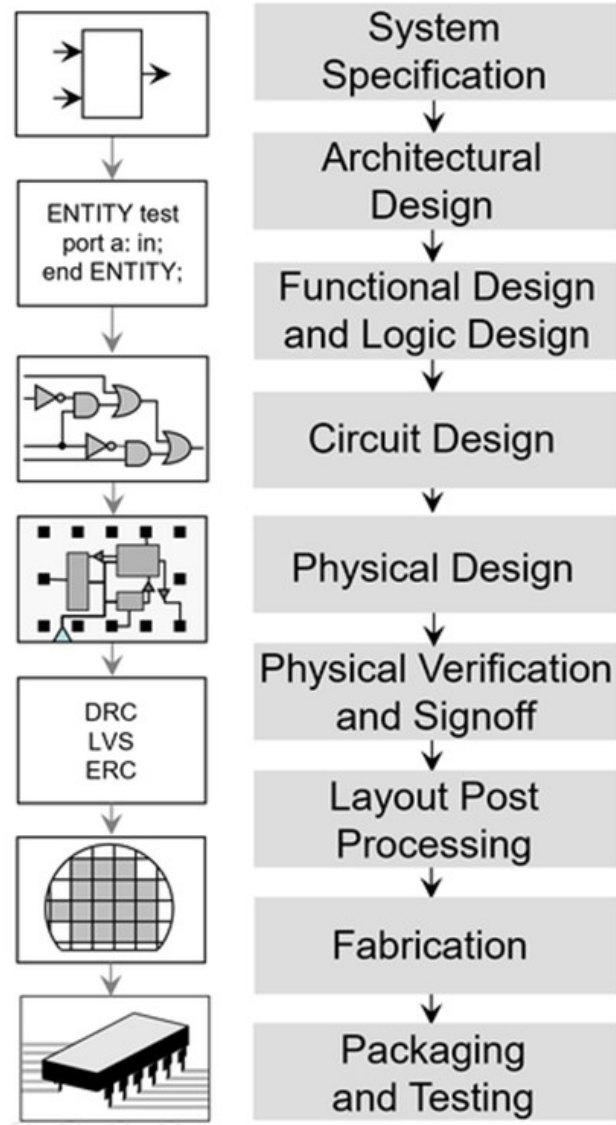


Flujo de Diseño Analógico

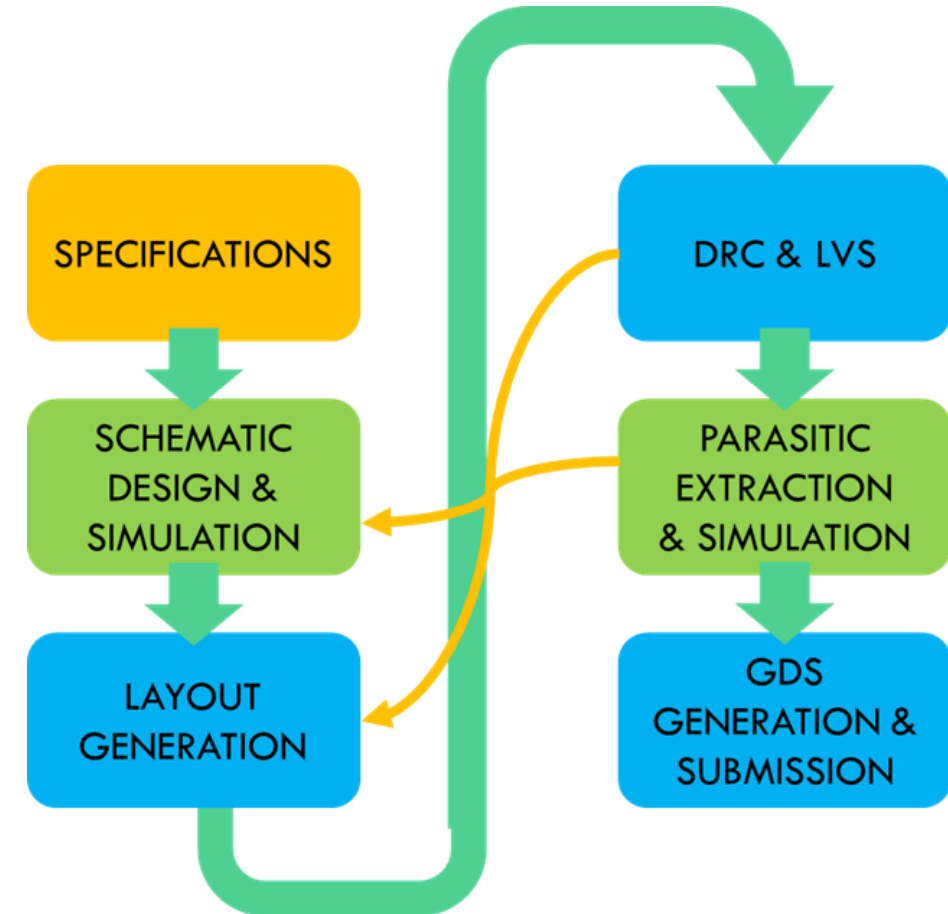


Flujo de Diseño

Digital



Analógico



¿Qué se necesita para diseñar un IC?

cādence[®]
SYNOPSYS[®]

Electronic
Design
Automation

EDA
Tools

HDL Simulation Logic Synthesis RTL Synthesis Floor Planning Power Planning Global Placement Detailed Placement	Global Routing Detailed Routing Static Code Analysis Static Timing Analysis Clock Tree Synthesis LVS	DFM RC Extraction DFT DRC
---	---	------------------------------------

ASIC
(Application-
Specific IC)

RTL
Design

Register Transfer
Logic

PDK
Data

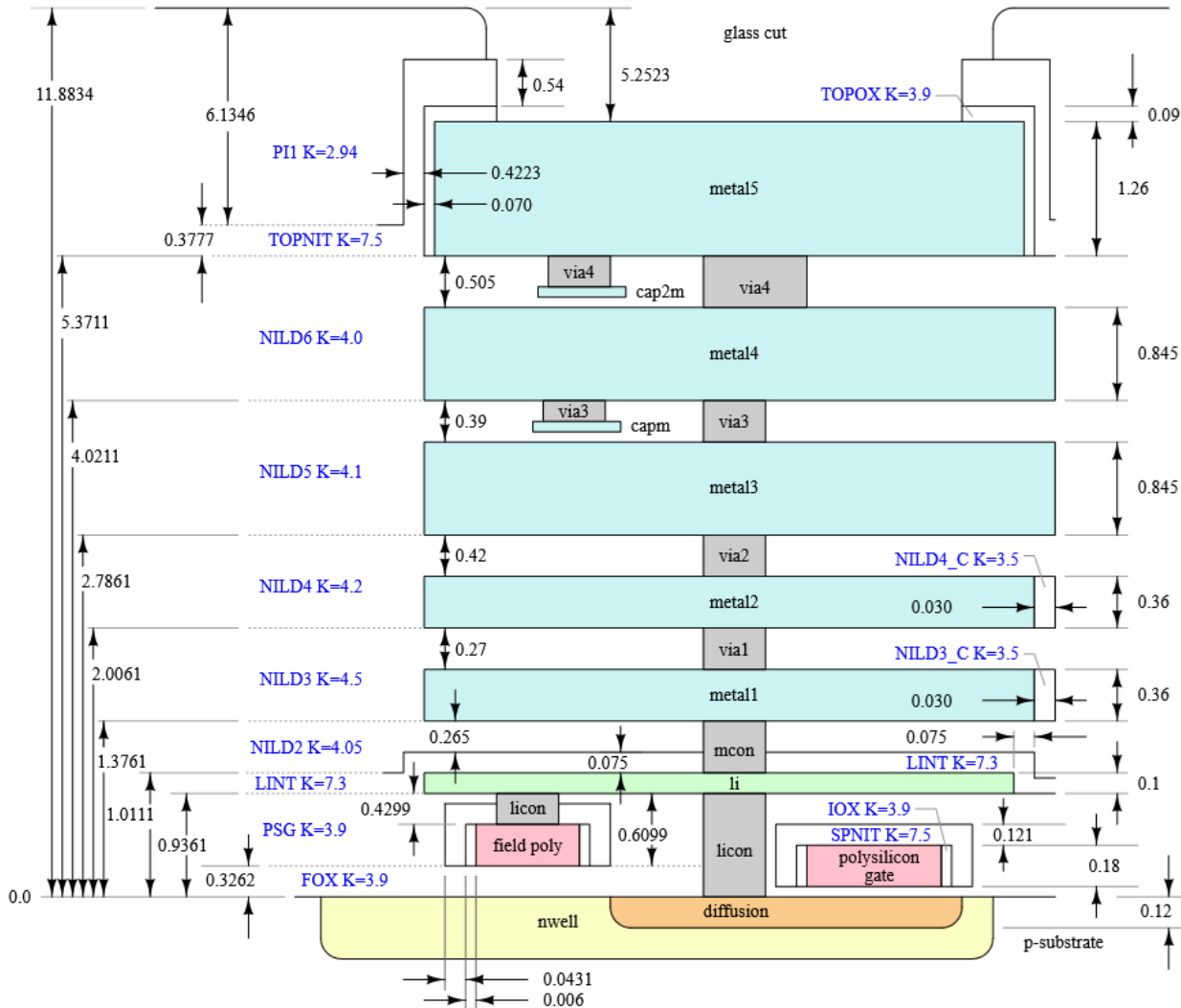
Process
Development Kit

Aquí empieza tu
diseño (Verilog)

librecores.org
opencores.org
github.com

...

¿Qué es un PDK?



Reglas de Diseño de Procesos

- DRC, LVS, PEX

Modelos de comportamiento

- Analógico
- Digital

Diseño analógico

- SPICE Models
- Parametric Cells

Diseño digital

- Standard Cells
- Timing Models

Datos extraídos

- SPICE/RC/LEF

Open PDKs



FOSS 130nm Production PDK
github.com/google/skywater-pdk

Lanzado en 2020



GlobalFoundries™
180nm

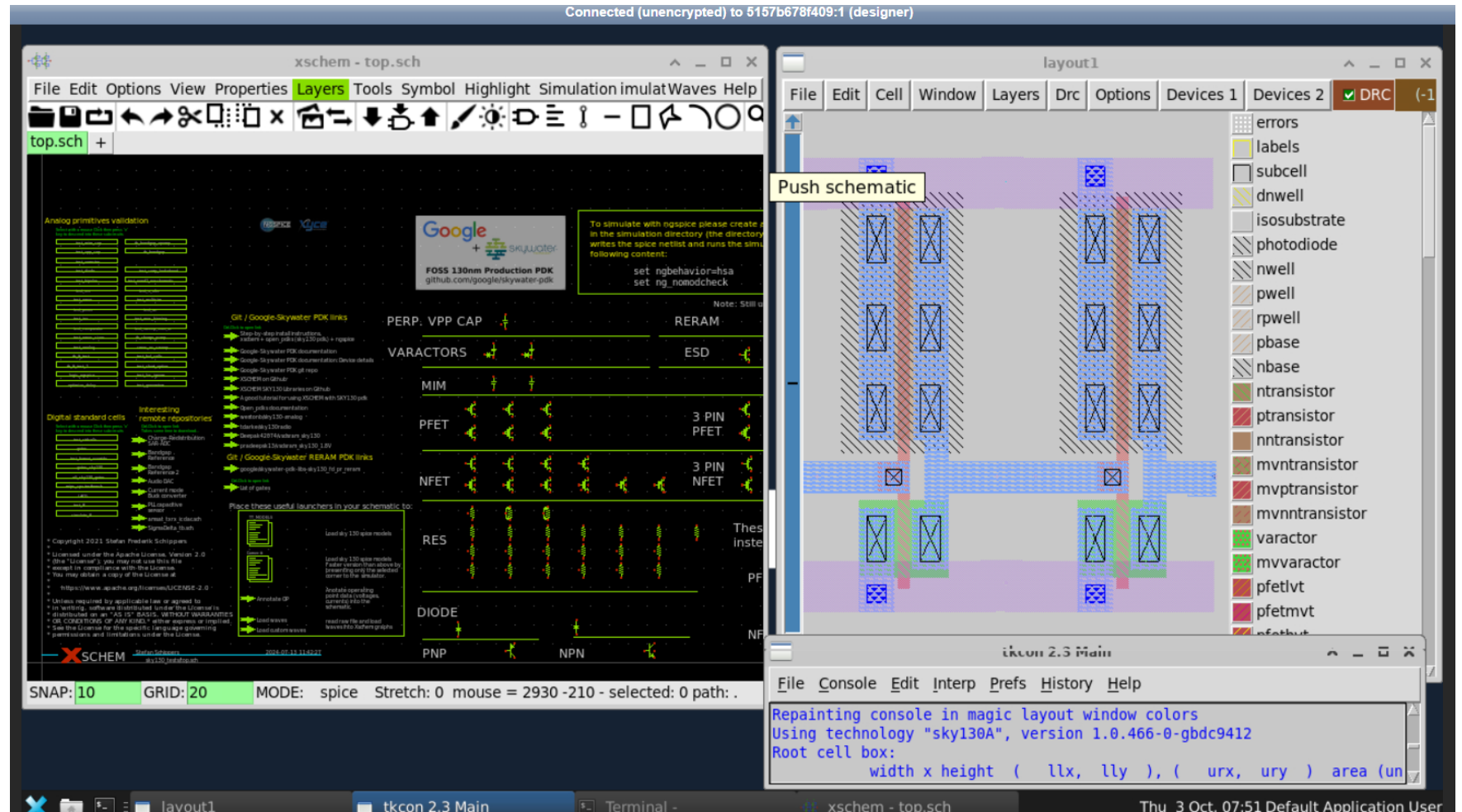


130nm SiGe BiCMOS

Open-source Tools



Institute for
Integrated Circuits
and Quantum
Computing



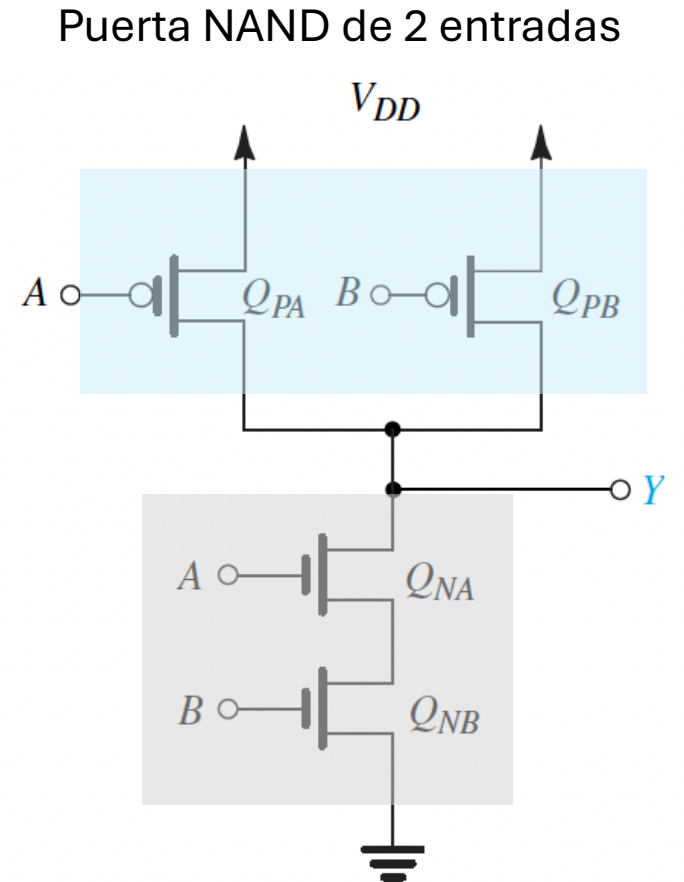
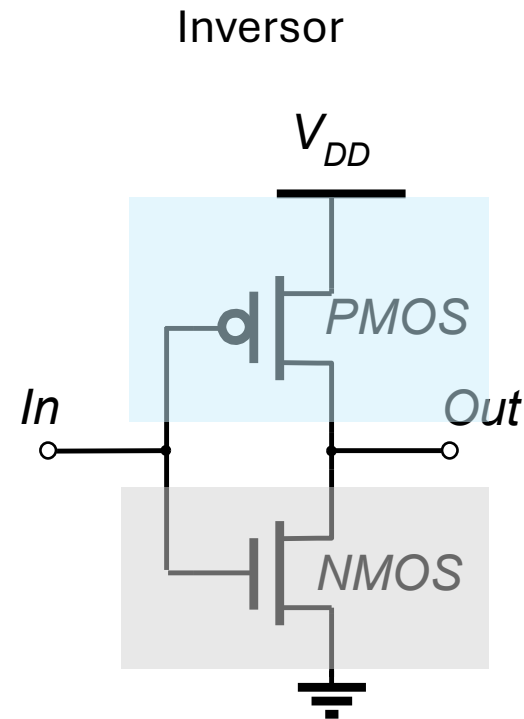
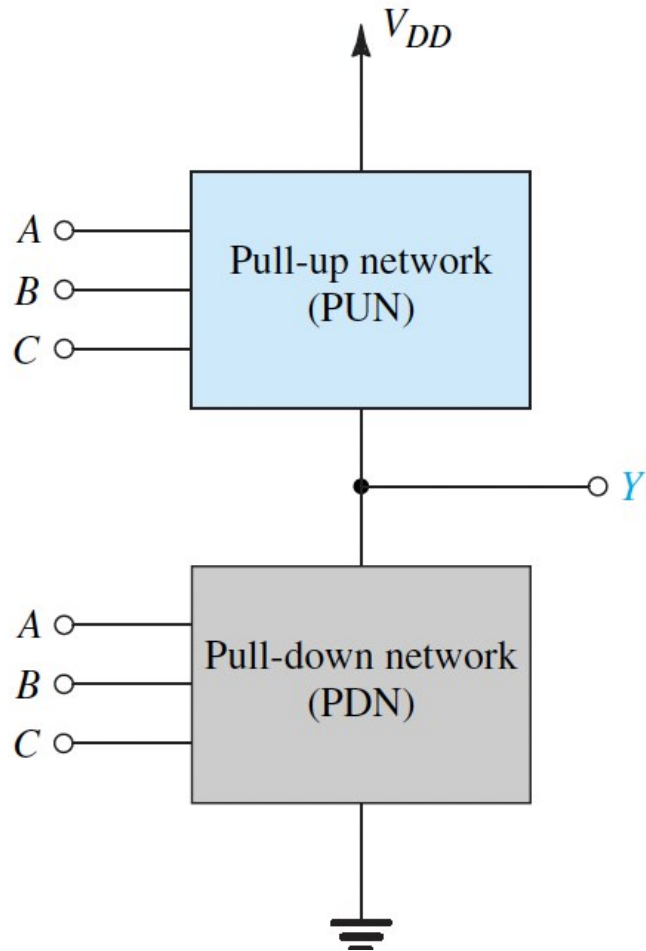
<https://github.com/iic-jku/IIC-OSIC-TOOLS>

Installation VIDEO:

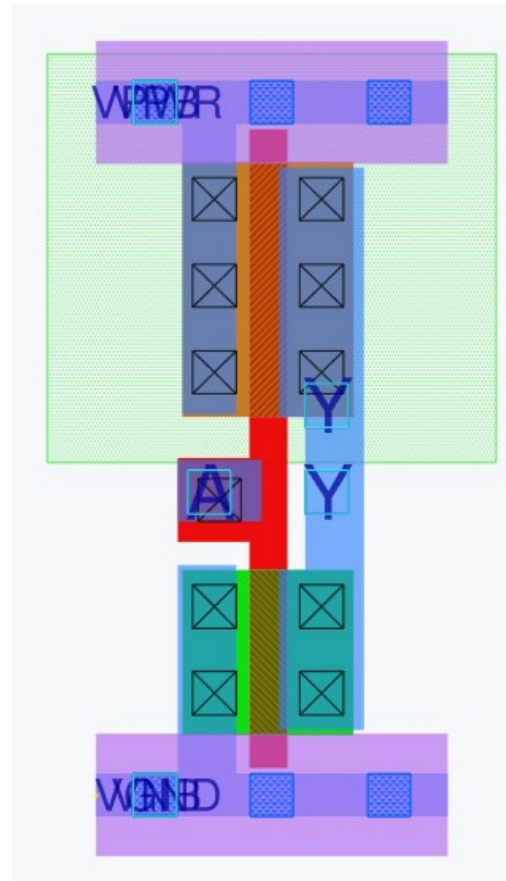
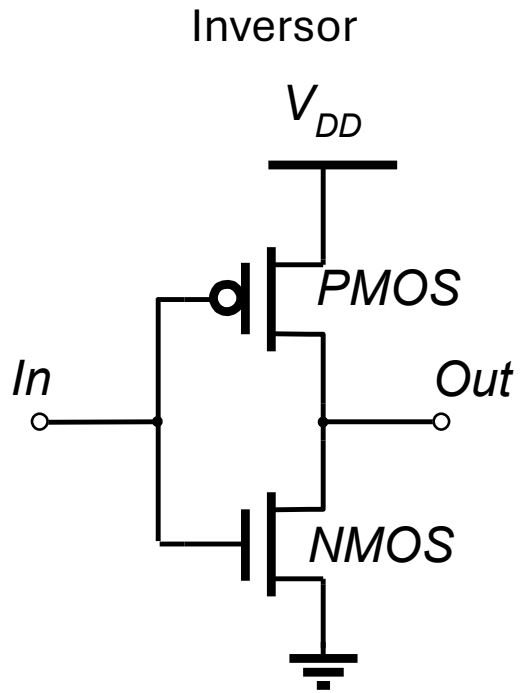
https://www.youtube.com/watch?v=-l_SrxKEHt0

Lógica CMOS

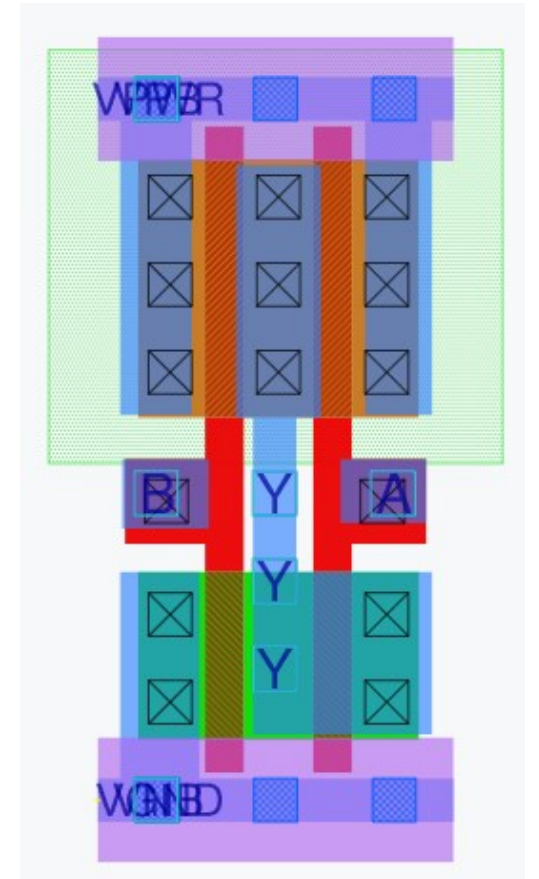
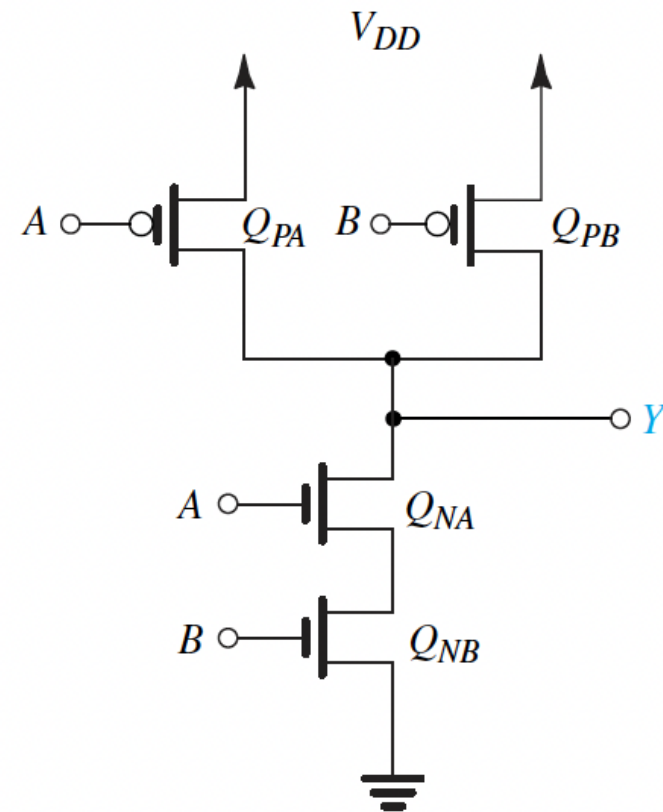
Estructura Básica Lógica CMOS



Celdas estándares (Standard cells)



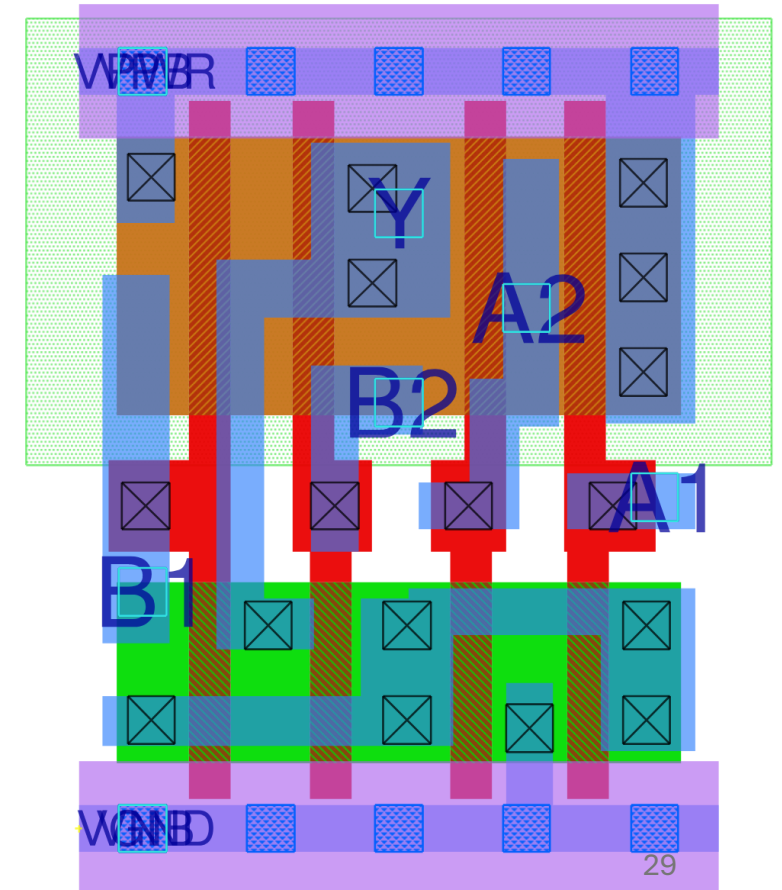
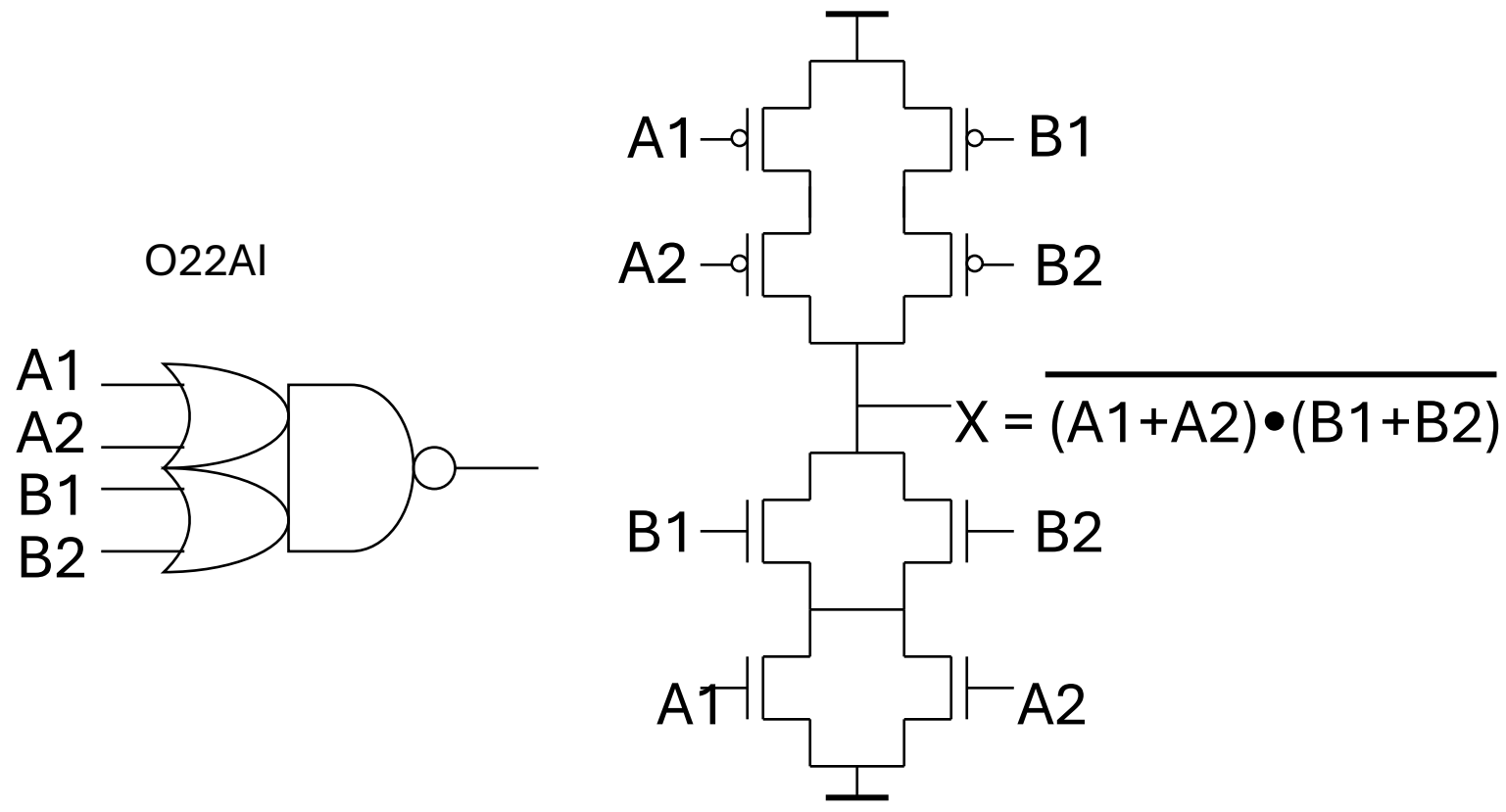
Puerta NAND de 2 entradas



Celdas estándares

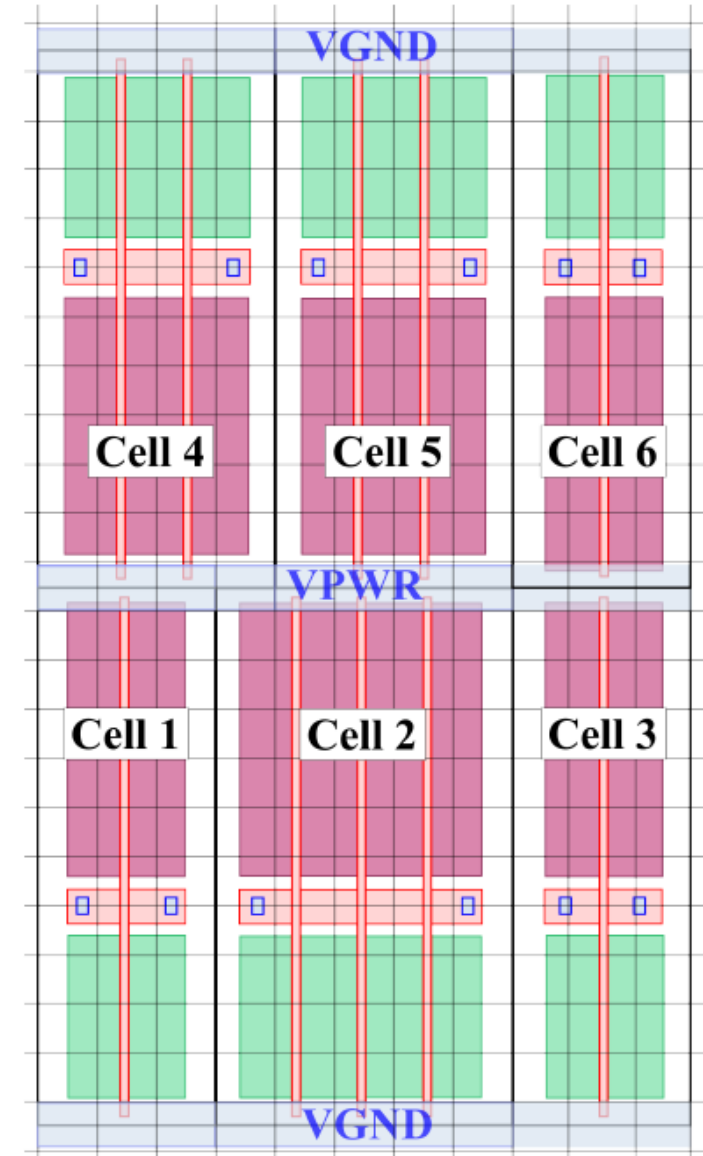
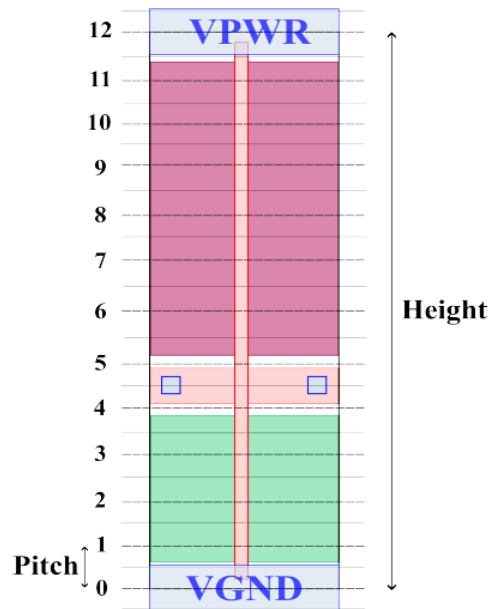
Celdas estándares en Skywater 130nm:

https://github.com/google/skywater-pdk-libs-sky130_fd_sc_hd/tree/main/cells/



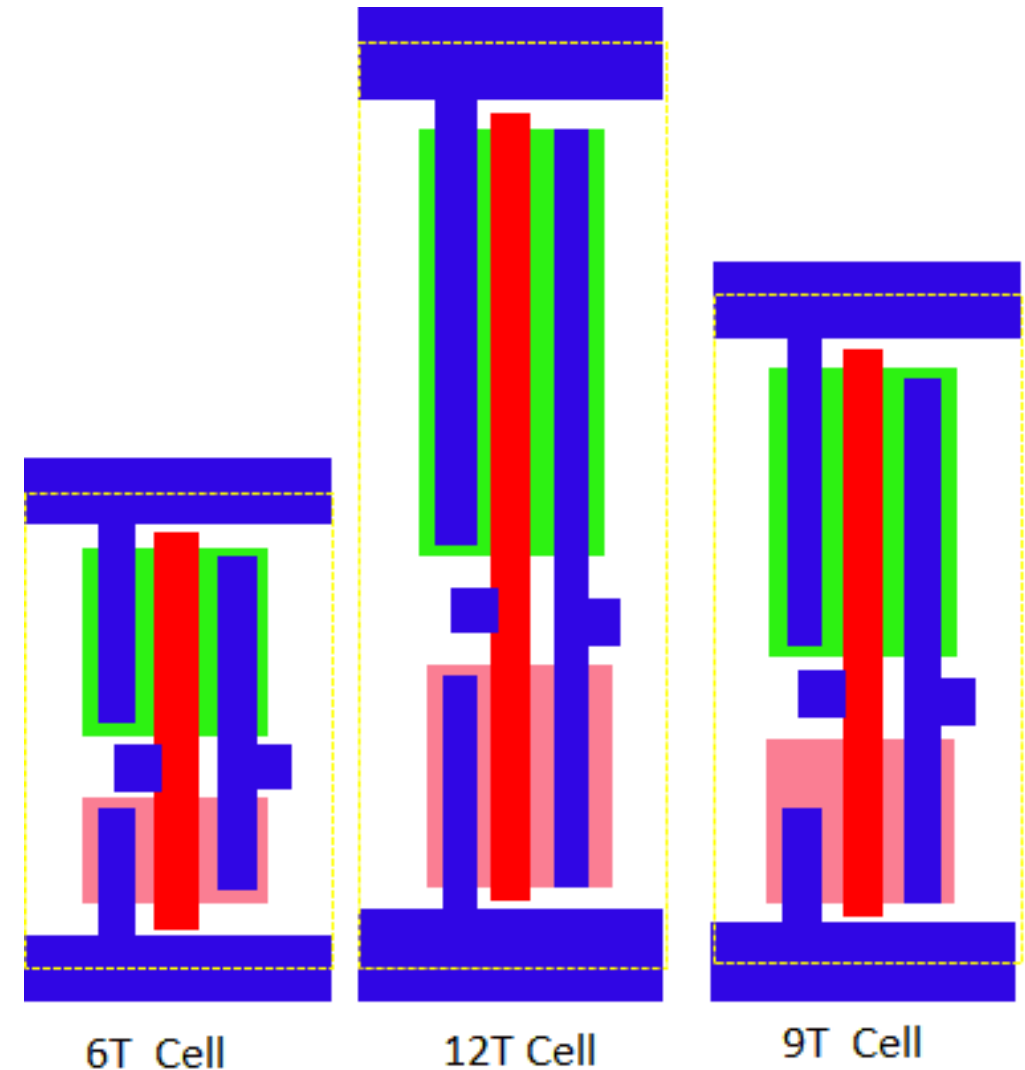
Standard Cell Library (SCL)

- Todas las celdas estándar tienen la misma altura (ladrillos LEGO)



Standard Cell Library (SCL)

- Una tecnología tiene celdas estándar para diferentes números de pistas:
 - Área
 - Energía
 - Velocidad
- Celdas en una SCL:
 - **Compuertas básicas:** NOT, AND, OR, NOR, NAND, XOR, etc.
 - **Compuertas complejas:** MUX, HA, FA, AOI, OAI, etc.
 - **Celdas Clock Tree :** Clock Buffers, Clock Inverters, ICG Cells
 - **Flips-Flops y Latches**
 - **Celdas especiales:** Fillers, Tap Cells, End-caps, De-caps

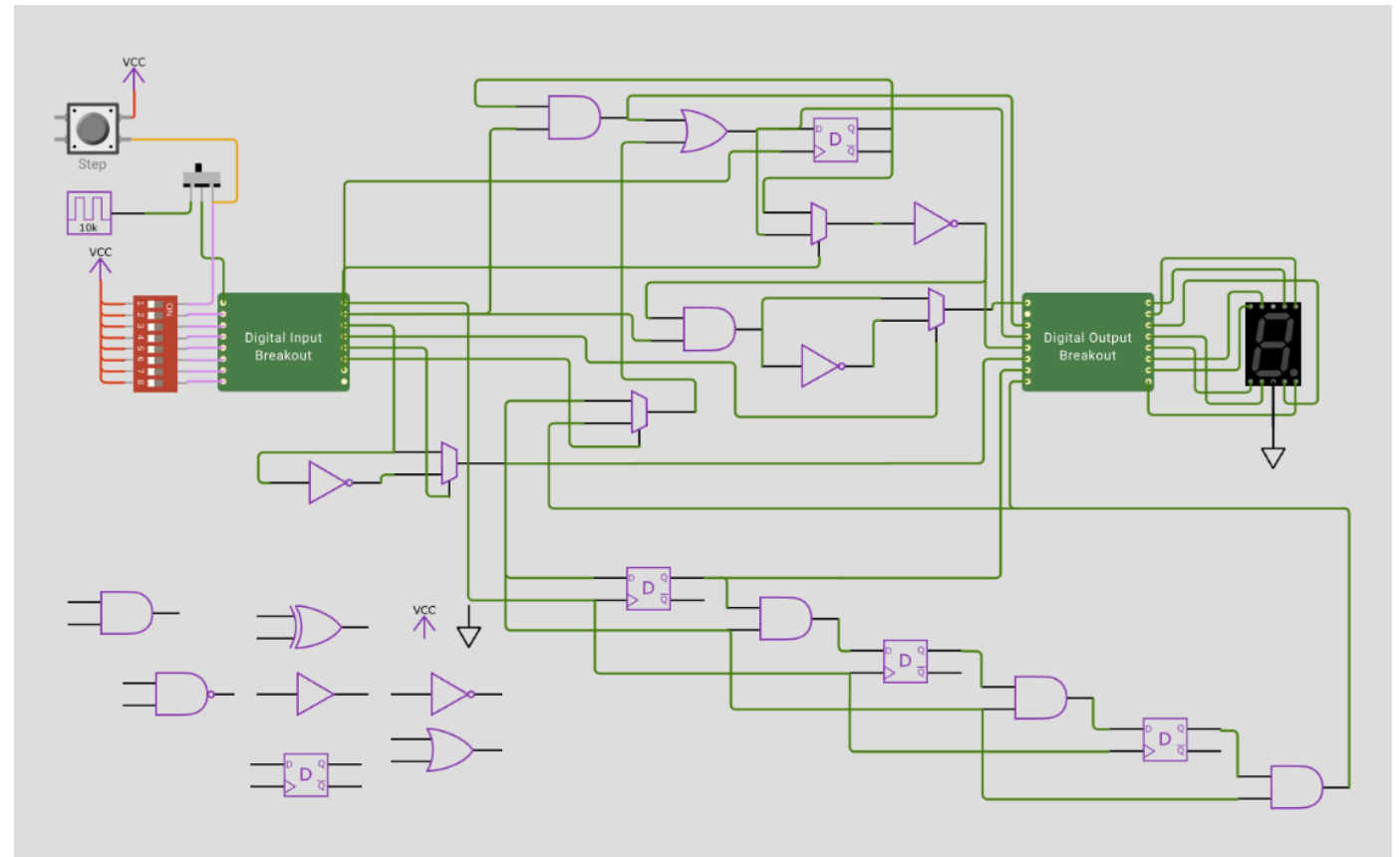


¡Crea un primer chip digital!

¡Crea un primer chip digital!

- Crea cuenta en:
 - wokwi.com
 - github.com
- Wokwi:
 - Simulator en línea
 - Chequea ERC
 - Exporta a un netlist Verilog
- Wokwi + Github → TinyTapeout
- Más ejemplos en https://tinytapeout.com/digital_design/

Plantilla: <https://wokwi.com/projects/354858054593504257>



¡Crea un primer chip digital!

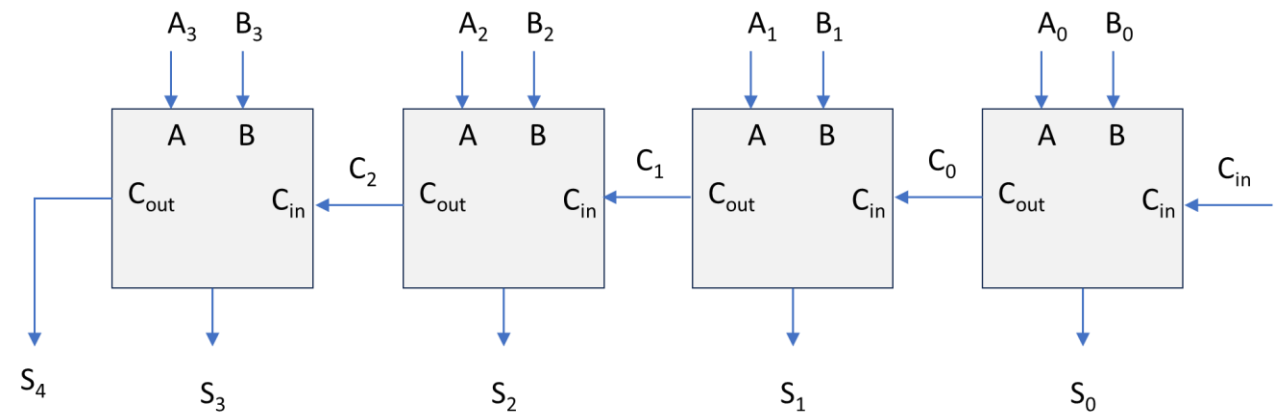


RETO 1

- Sumador de 4 bits con acarreo de entrada y salida

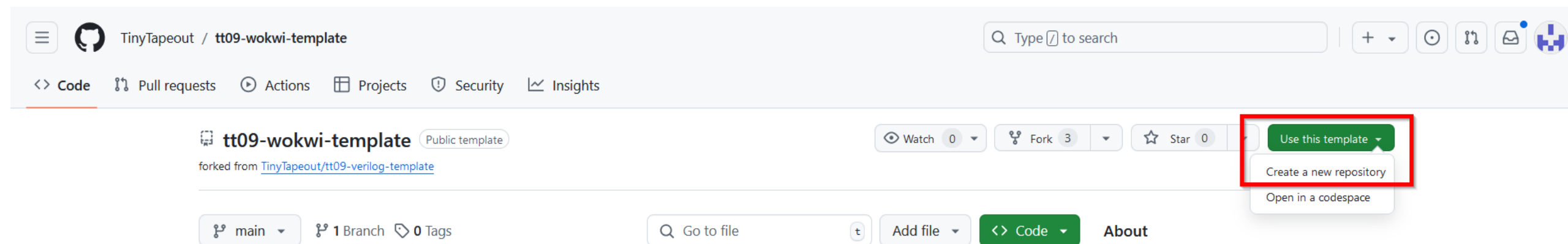
$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A \oplus B)$$



¡Crea un primer chip digital!

- Crear el layout:
 - Usar la plantilla <https://github.com/TinyTapeout/ttsky-wokwi-template>



¡Crea un primer chip digital!

- Modificar las propiedades del repositorio

The screenshot displays the GitHub repository settings for 'deepbits/deepsita'. The top navigation bar includes links for Code, Pull requests, Actions, Zenhub, Projects, Wiki, Security, Insights, and Settings (marked with a red box and '1'). The left sidebar shows various settings categories: General, Access, Collaborators, Moderation options, Code and automation, and Pages (marked with a red box and '2'). The main content area is titled 'GitHub Pages' and includes a description, a live site URL (<https://deepsita.github.io/tt09-wokwi-test/>), and a 'Visit site' button. Below this, the 'Build and deployment' section is shown with the 'Source' tab selected (marked with a red box and '3'). The 'Source' tab lists two options: 'GitHub Actions' (checked, marked with a red box and '4') and 'Deploy from a branch'. The 'GitHub Actions' option is described as 'Best for using frameworks and customizing your build process'. The 'Deploy from a branch' option is described as 'Classic Pages experience'. The page also includes a 'Custom domains' section with a 'Save' button and a 'Remove' button.

Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Your site is live at <https://deepsita.github.io/tt09-wokwi-test/>
Last deployed by [deepsita](#) 18 hours ago

Visit site

Build and deployment

Source

GitHub Actions

✓ GitHub Actions
Best for using frameworks and customizing your build process

Deploy from a branch
Classic Pages experience

Custom domains.

Save Remove

¡Crea un primer chip digital!

- Modificar info.yaml usando poniendo en código wokwi_id el número de su proyecto, completar demás campos y hacer commit

The screenshot shows the Wokwi IDE interface with a modal dialog for committing changes. The background editor shows the `info.yaml` file for a project named `tt09-wokwi-test` on the `main` branch. The `info.yaml` file contains the following content:

```
1 # Tiny Tapeout project information
2 project:
3   wokwi_id: 4092073136379
4   title: "Encoder"
5   author: "Skandha"
6   discord: ""
7   description: "8x3 encoder"
8   language: "Wokwi"
9   clock_hz: 0
10
11 # How many tiles your design uses
12 tiles: "1x1" # Value must be a power of 2
13
14 # The pinout of your project.
15 pinout:
16   # Inputs
17   ui[0]: "IN0"
18   ui[1]: "IN1"
19   ui[2]: "IN2"
20   ui[3]: "IN3"
21   ui[4]: "IN4"
22   ui[5]: "IN5"
```

The `Commit changes` dialog box is open, showing the following fields and options:

- Commit message:** A text input field containing "Update info.yaml".
- Extended description:** A text area with the placeholder text "Add an optional extended description..".
- Commit options:** Two radio buttons are present:
 - ☒ Commit directly to the main branch
 - ☐ Create a new branch for this commit and start a pull request [Learn more about pull requests](#)
- Buttons:** At the bottom of the dialog are "Cancel" and "Commit changes" buttons. The "Commit changes" button is highlighted with a red box and a red circle with the number 2.

In the top right corner of the editor, there are two buttons: "Cancel changes" and "Commit changes...". The "Commit changes..." button is highlighted with a red box and a red circle with the number 1.

¡Crea un primer chip digital!

- Modificar info.yaml usando poniendo en código wokwi_id el número de su proyecto, completar demás campos y hacer commit

The screenshot shows the Wokwi IDE interface with the 'info.yaml' file open. A 'Commit changes' dialog box is displayed in the center. The dialog has a title bar 'Commit changes' and a close button. It contains a 'Commit message' field with the text 'Update info.yaml', an 'Extended description' field with the placeholder 'Add an optional extended description..', and two radio buttons for commit options: 'Commit directly to the main branch' (selected) and 'Create a new branch for this commit and start a pull request' (with a link to 'Learn more about pull requests'). At the bottom of the dialog are 'Cancel' and 'Commit changes' buttons. A red box with a '1' highlights the 'Commit changes...' button in the top right of the editor. Another red box with a '2' highlights the 'Commit changes' button at the bottom of the dialog. The background shows the 'info.yaml' file with the following content:

```
1 # Tiny Tapeout project information
2 project:
3   wokwi_id: 4092073136379
4   title: "Encoder"
5   author: "Skandha"
6   discord: " "
7   description: "8x3 encoder"
8   language: "Wokwi"
9   clock_hz: 0
10
11 # How many tiles your design uses
12 tiles: "1x1" # Valid values are 1x1, 2x2, 4x4, 8x8, 16x16
13
14 # The pinout of your project.
15 pinout:
16   # Inputs
17   ui[0]: "IN0"
18   ui[1]: "IN1"
19   ui[2]: "IN2"
20   ui[3]: "IN3"
21   ui[4]: "IN4"
22   ui[5]: "IN5"
```


¡Crea un primer chip digital!

- Revisa tu diseño en el menú Actions

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

Settings

Summary

Jobs

- gds
- precheck
- viewer

Run details

- Usage
- Workflow file

- Warning: Selection "sky130_fd_sc_hd_comb_1" did not match any module.
- Warning: Selection "LO" did not match any object.

Routing stats

Utilisation (%)	Wire length (um)
0.94	443

Cell usage by Category

Category	Cells	Count
Fill	decap fill	1467
Tap	tappwrvgnd	225
Buffer	buf clkbuf	31
Misc	conb	21
OR	or4	3

55 total cells (excluding fill and tap cells)

Job summary generated at run-time

Summary 001110100

sky130_fd_sc_hd_or4

sky130_fd_sc_hd_or4 symbols

sky130_fd_sc_hd_or4 schematic

sky130_fd_sc_hd_or4 GDSII layouts

tt_um_wokwi_409207313637927937

- KEYS
- 1: Hide Fill, Decap, Tap cells
 - 2: Hide top cell geometry
 - 3: Isolate selection / back
 - 4: Zoom to selection



Controls

View Settings

Layers

Cells/Instances

ALL

Sort By

Name

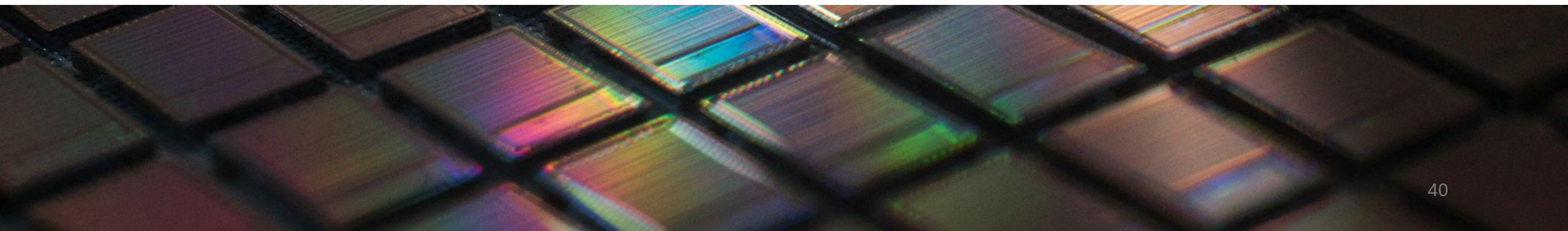
Cell types: 13 - Instances: 1747

- sky130_ef_sc_hd_decap_12 (x913)
- sky130_fd_sc_hd_buf_2 (x21)
- sky130_fd_sc_hd_clkbuf_1 (x7)
- sky130_fd_sc_hd_clkbuf_4 (x3)
- sky130_fd_sc_hd_comb_1 (x21)
- sky130_fd_sc_hd_decap_3 (x103)
- sky130_fd_sc_hd_decap_4 (x23)
- sky130_fd_sc_hd_decap_5 (x182)
- sky130_fd_sc_hd_decap_8 (x3)
- sky130_fd_sc_hd_fill_1 (x206)
- sky130_fd_sc_hd_fill_2 (x37)
- sky130_fd_sc_hd_or4_1 (x3)
- sky130_fd_sc_hd_tappwrvgnd_1 (x225)

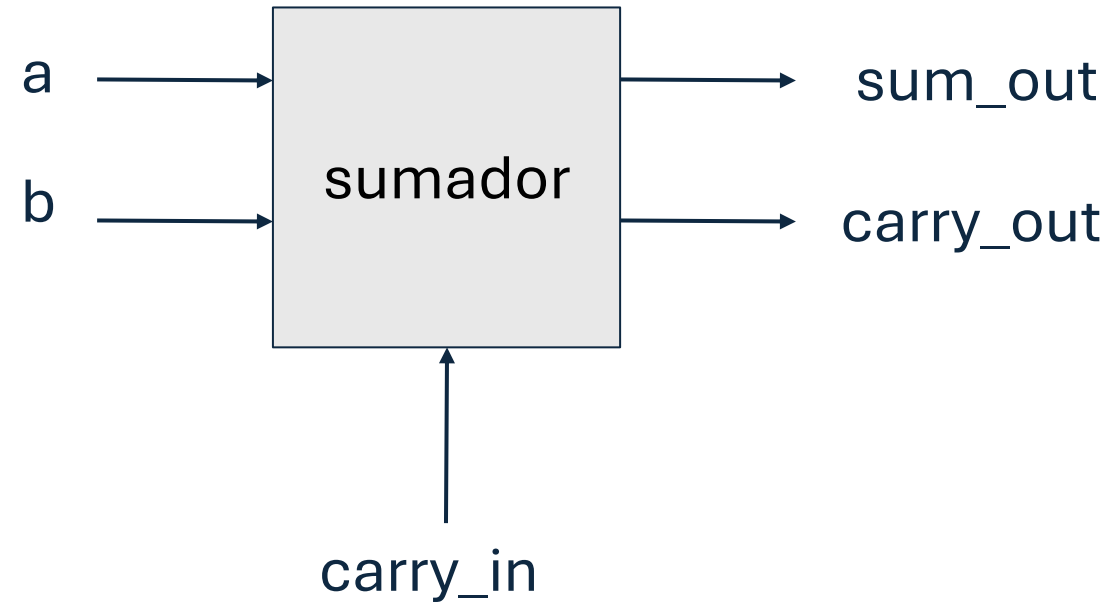
Performance

Experimental

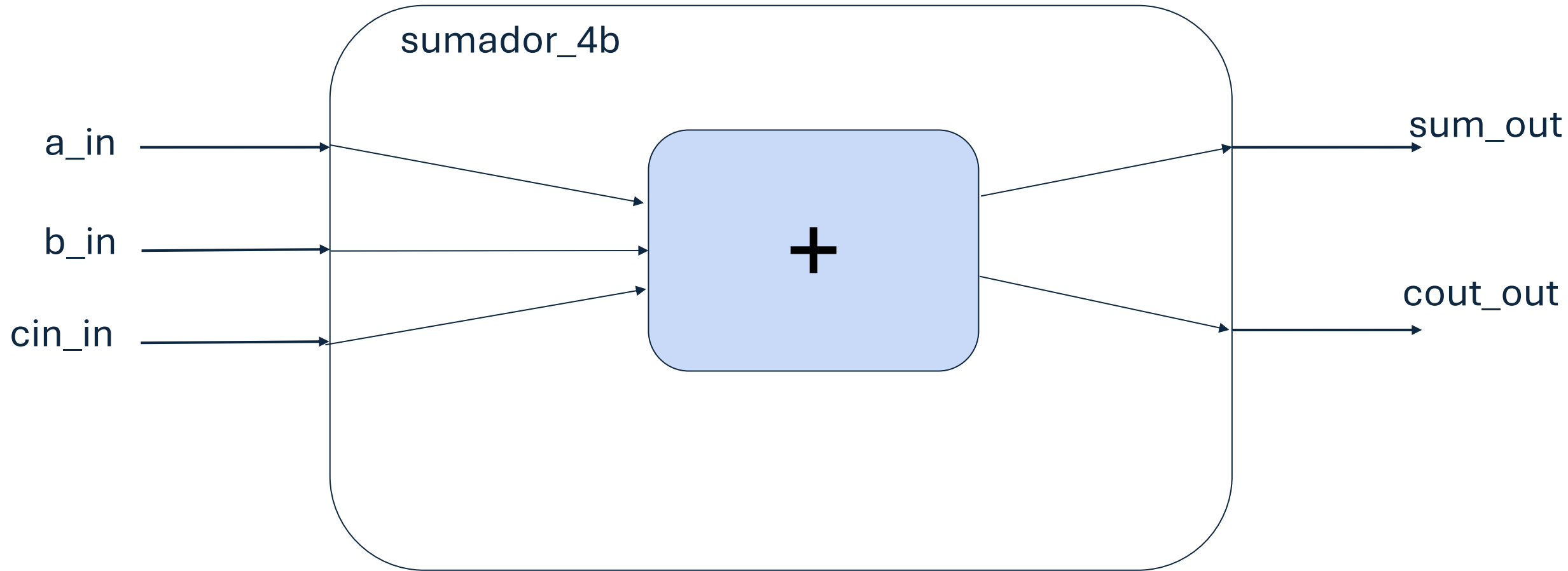
¿Cómo diseñar un chip más complejo?



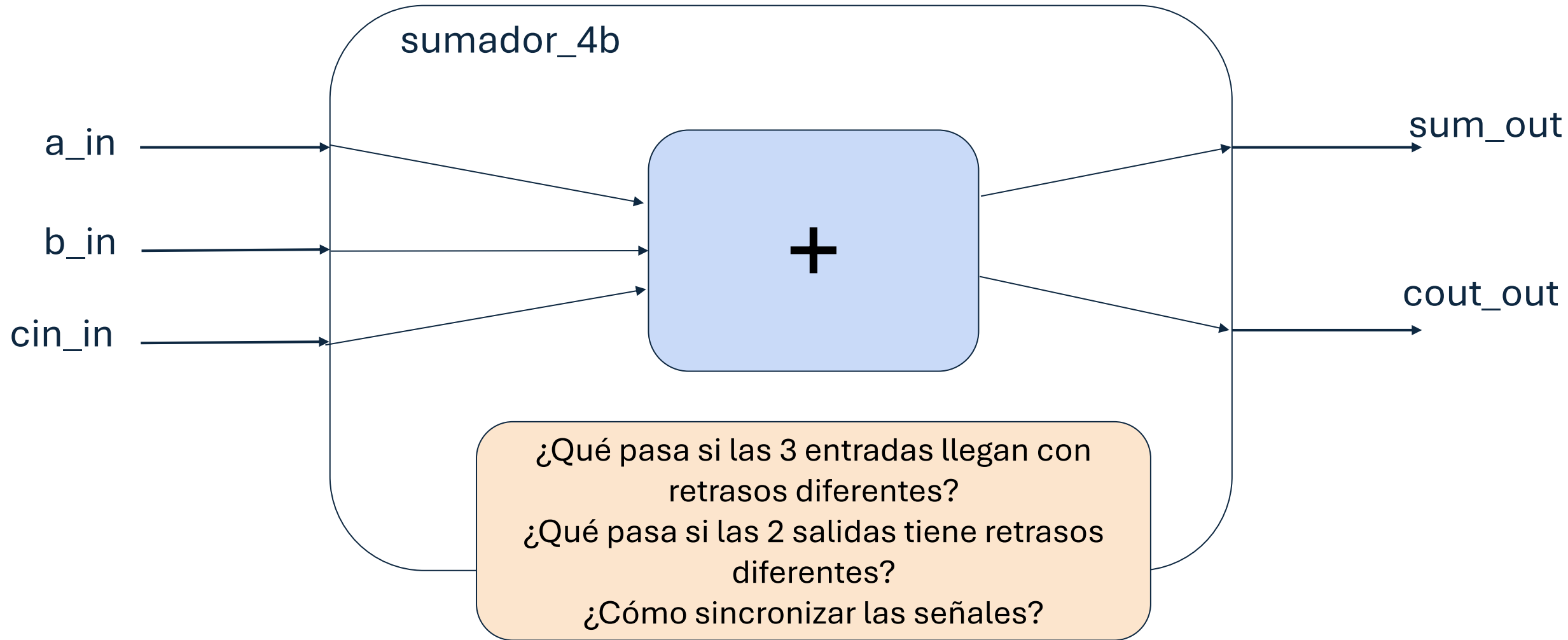
Ejemplo Sumador (Versión HDL)



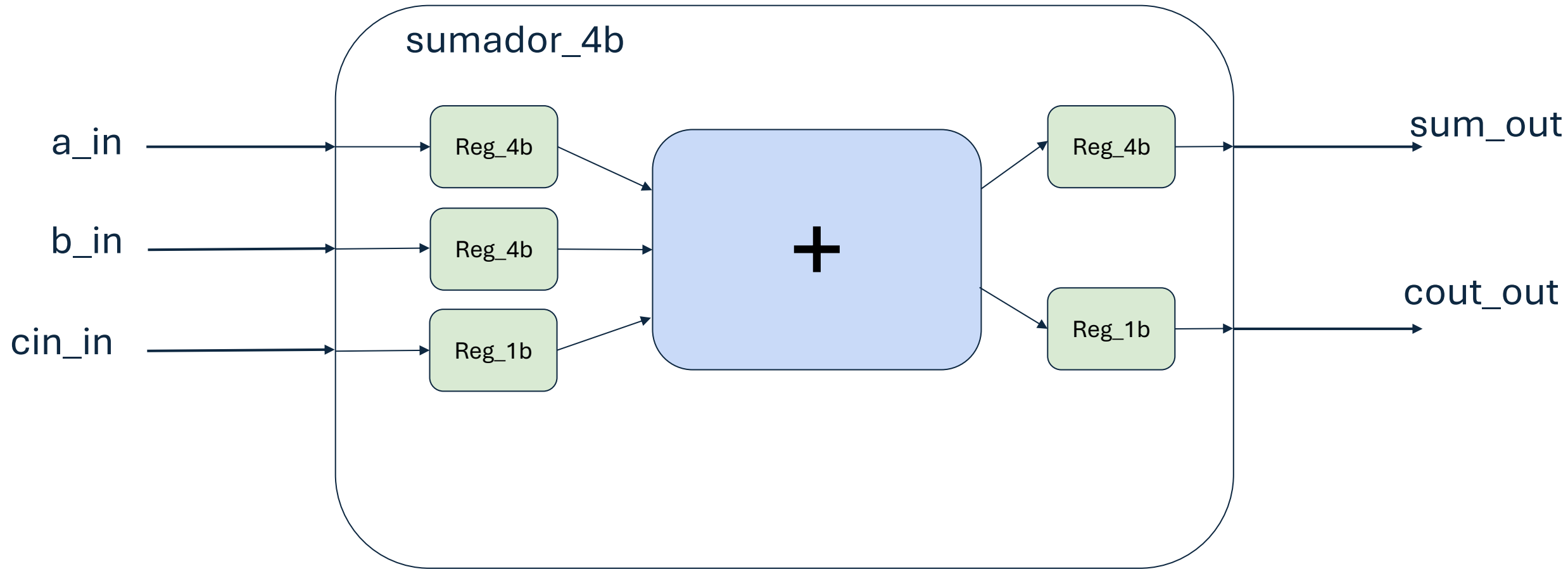
Ejemplo Sumador



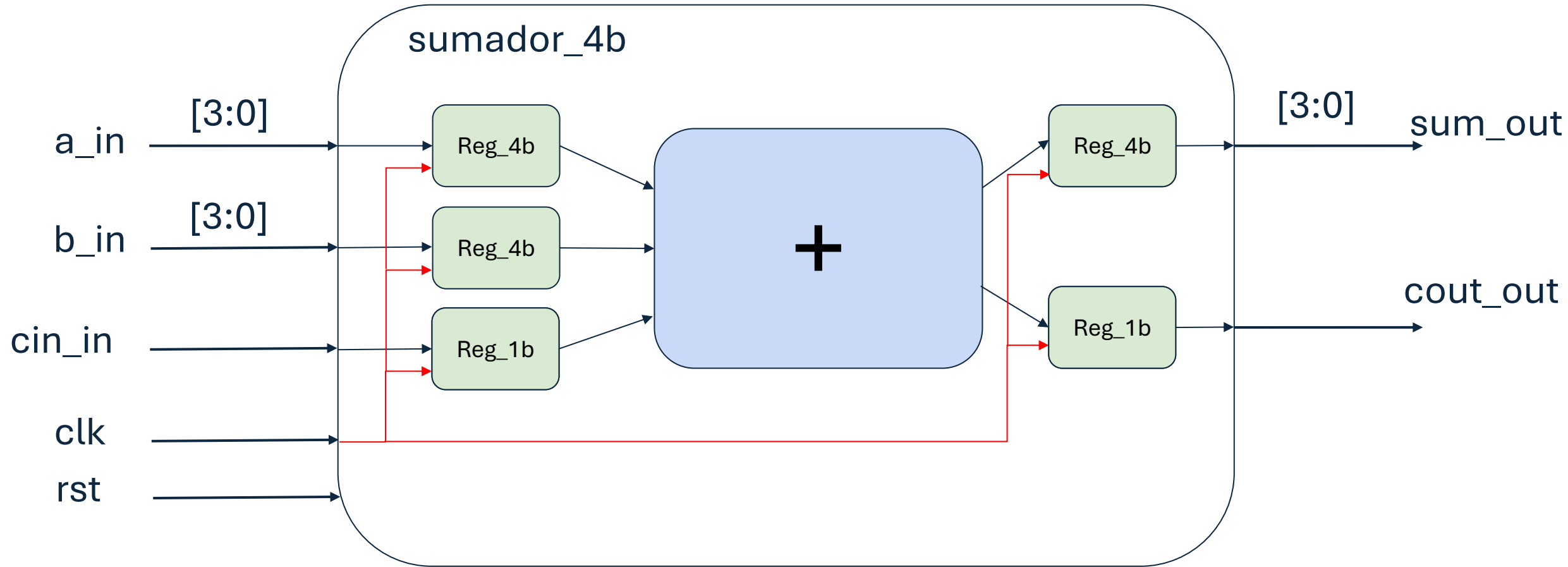
Ejemplo Sumador



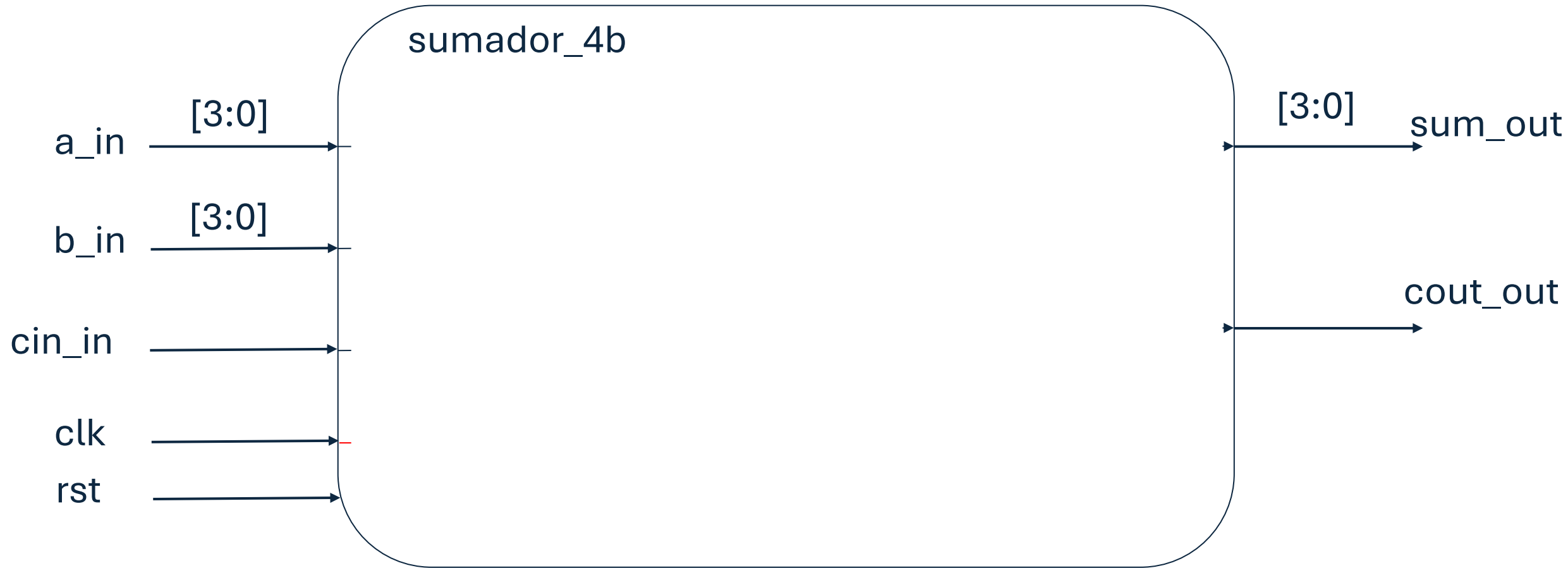
Ejemplo Sumador



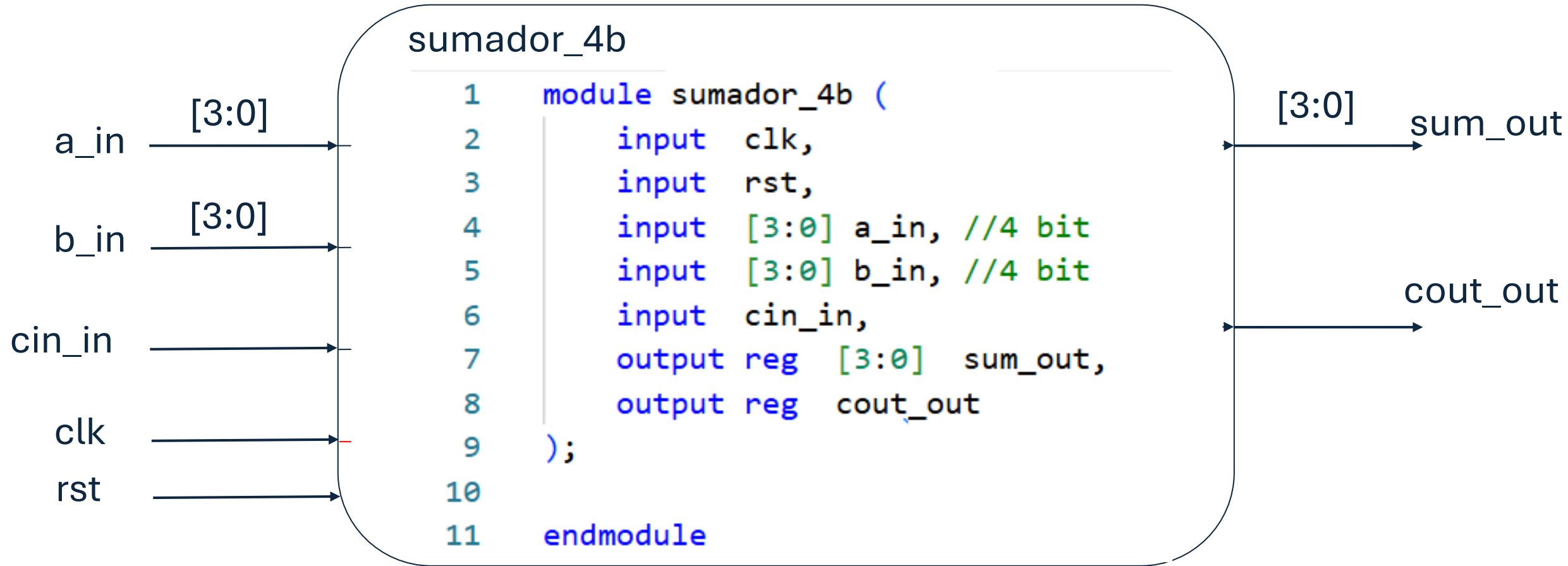
Ejemplo Sumador



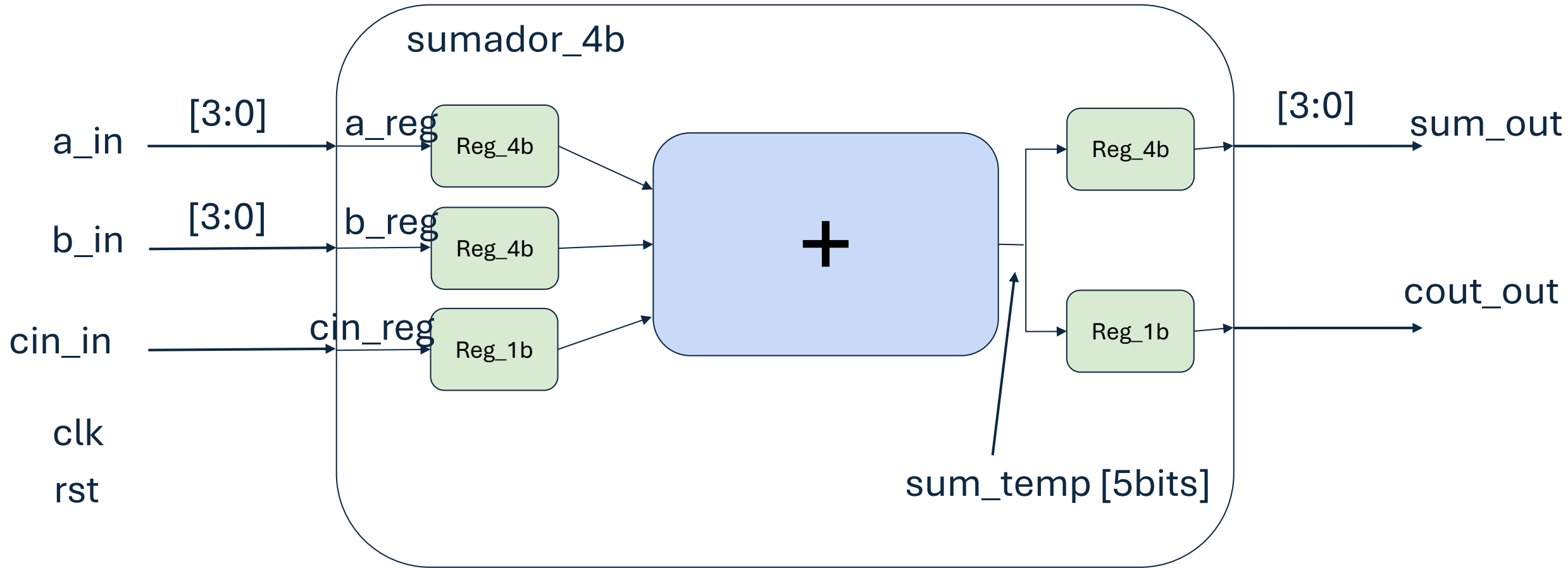
Ejemplo Sumador



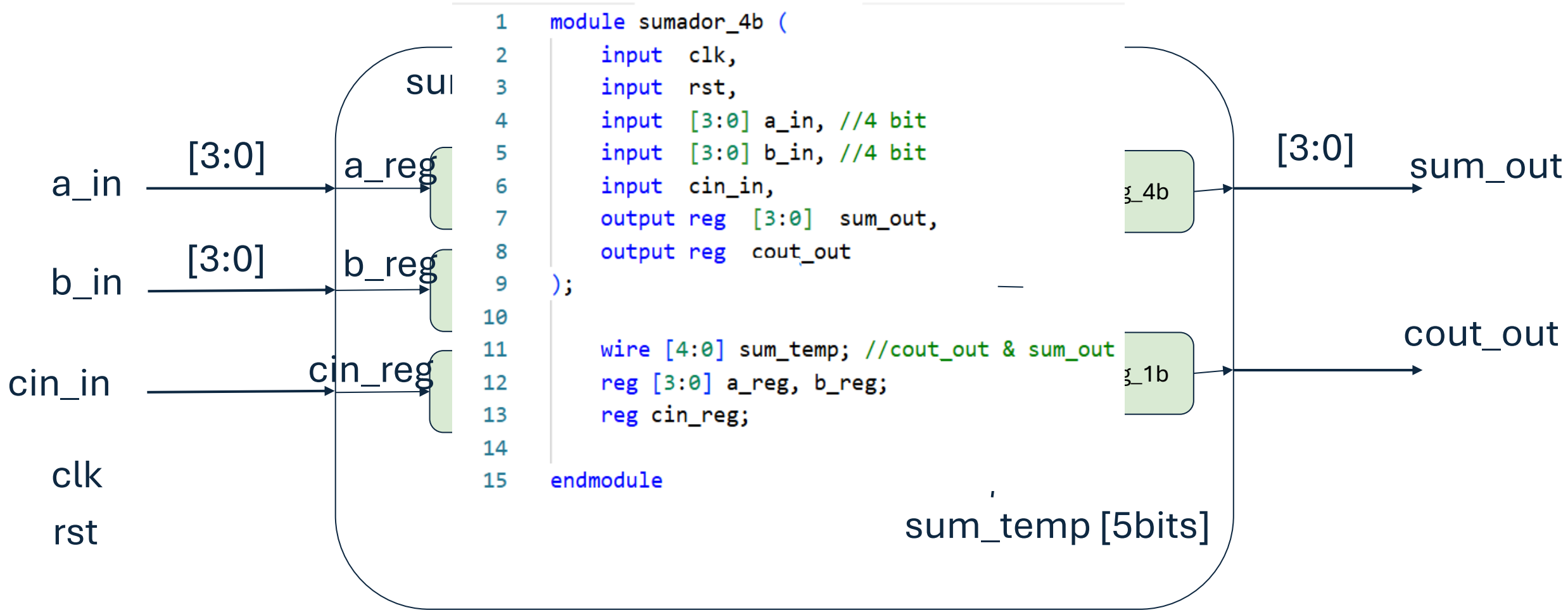
Ejemplo Sumador



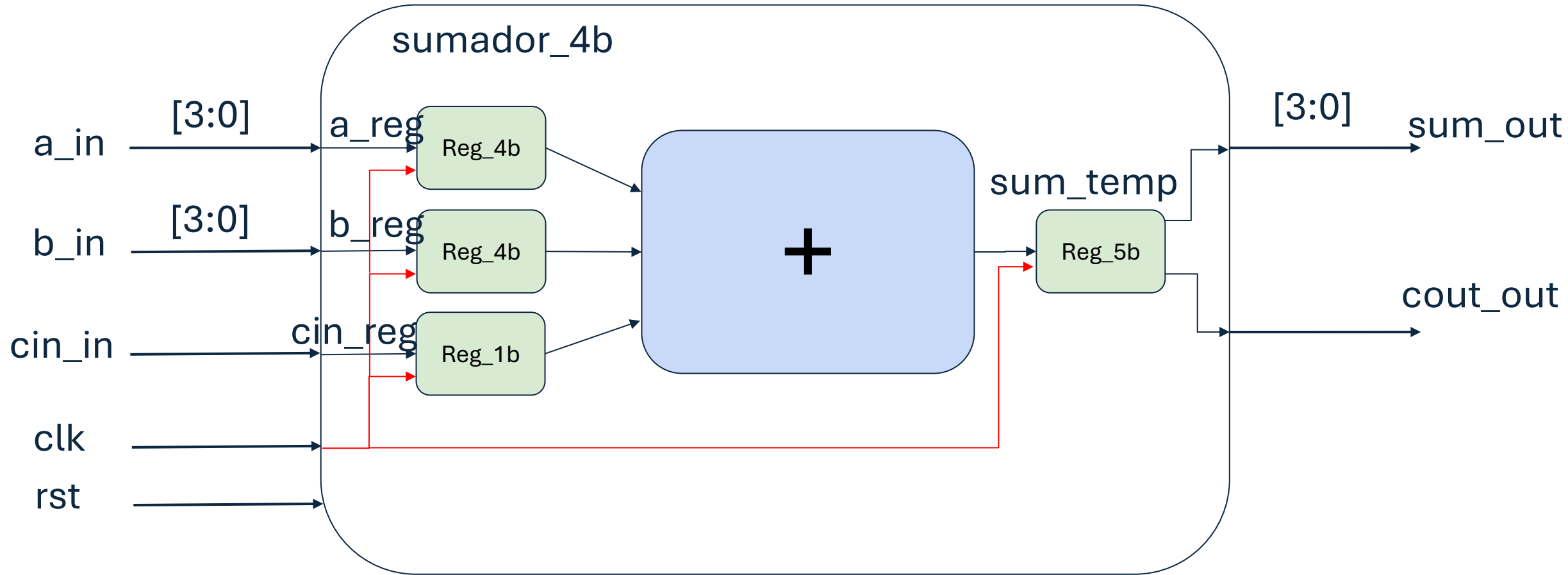
Ejemplo Sumador



Ejemplo Sumador



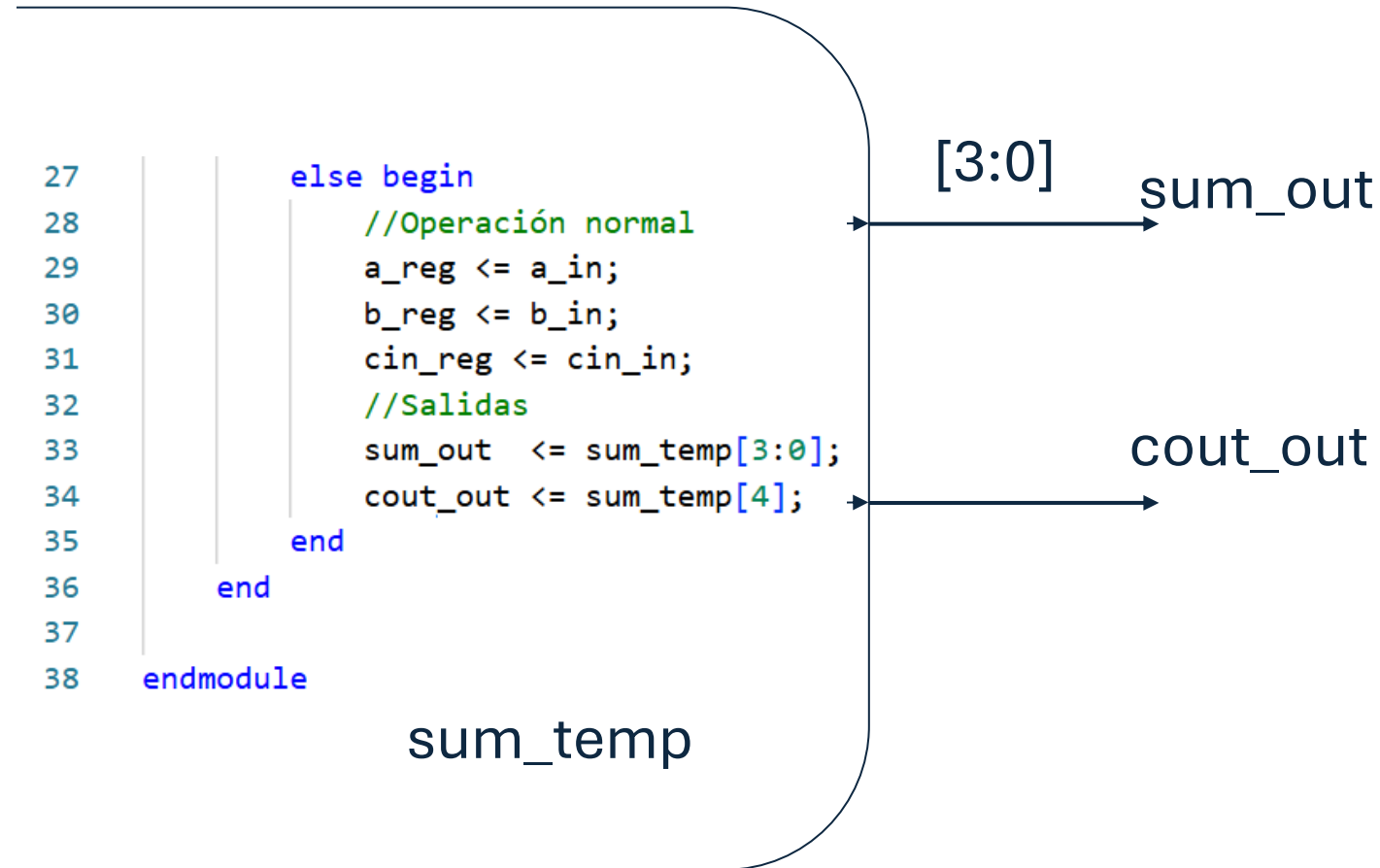
Ejemplo Sumador



Ejemplo Sumador

```
1  module sumador_4b (  
2      input  clk,  
3      input  rst,  
4      input  [3:0] a_in, //4 bit  
5      input  [3:0] b_in, //4 bit  
6      input  cin_in,  
7      output reg [3:0] sum_out,  
8      output reg  cout_out  
9  );  
10  
11      wire [4:0] sum_temp; //cout_out & sum_out  
12      reg [3:0] a_reg, b_reg;  
13      reg cin_reg;  
14  
15      // Operación combinacional del sumador  
16      assign sum_temp = a_reg + b_reg + cin_reg;  
17  
18      always @(posedge clk) begin  
19          if (rst) begin  
20              //Reset sincrónico  
21              a_reg <= 4'b0;  
22              b_reg <= 4'b0;  
23              cin_reg <= 1'b0;  
24              sum_out <= 4'b0;  
25              cout_out <= 1'b0;  
26          end
```

```
27      else begin  
28          //Operación normal  
29          a_reg <= a_in;  
30          b_reg <= b_in;  
31          cin_reg <= cin_in;  
32          //Salidas  
33          sum_out  <= sum_temp[3:0];  
34          cout_out <= sum_temp[4];  
35      end  
36  end  
37  
38  endmodule
```



Flujo de diseño digital *semicustom*

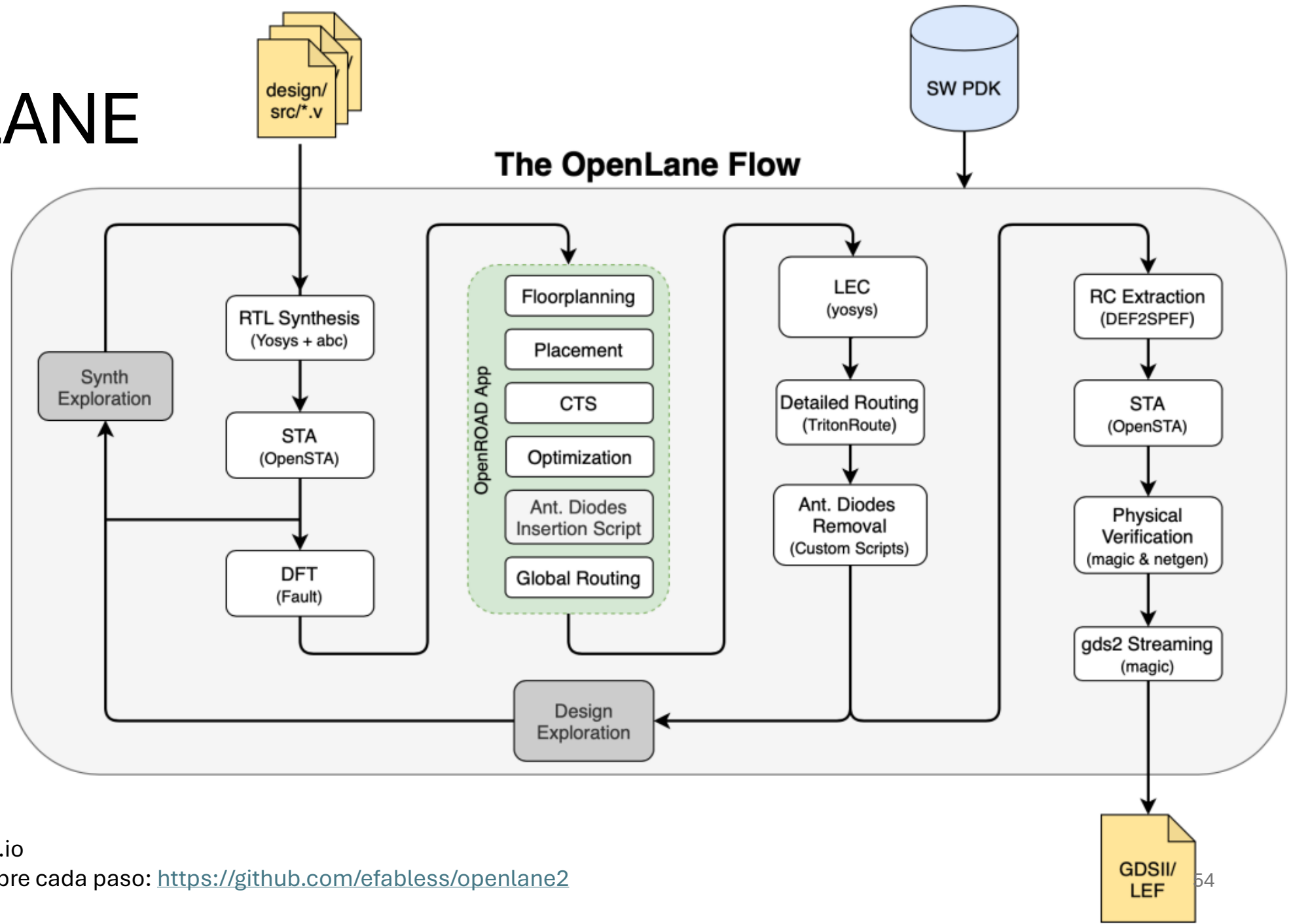
Diseño digital *semicustom*



OpenLANE (ahora LibreLANE)

- Basado en el proyecto OpenROAD
- Produce un GDSII sin intervención humana
- Compatible con PDKs: SkyWater130, GlobalFoundries180, IHP130

OpenLANE



Tomado de openlane.io

Para más detalles sobre cada paso: <https://github.com/efabless/openlane2>

Primeros pasos en OpenLANE

- Clone el repositorio:

```
git clone https://github.com/jimarin/taller_crea_chip
```

- Compruebe que las variables de entorno **PDK** y **PDK_ROOT** estén configuradas en **sky130A** y **/foss/pdks**, respectivamente, utilizando:

```
echo $PDK
```

```
echo $PDK_ROOT
```

- Si no está configurado, use

```
sak-pdk sky130A
```

Primeros pasos en OpenLANE

- Para Síntesis:

librelane config.json

Estructura de carpetas

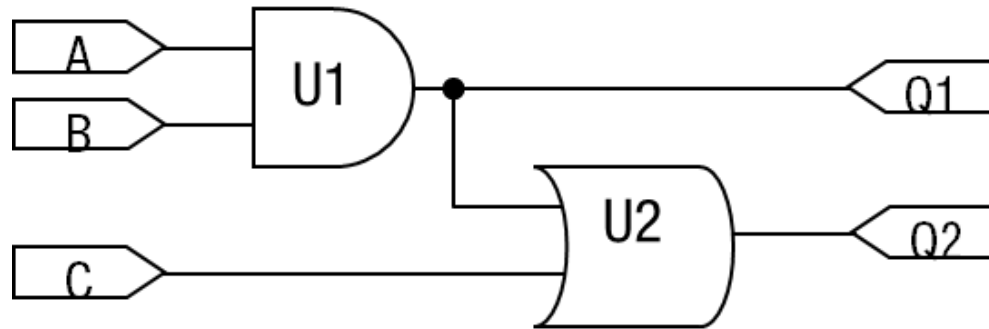
```
config.json
pin_order.cfg
src/
    [Archivos Verilog]
    [Archivos SDC]
runs/
    [1 carpeta para cada ejecución]
```

Resultados finales en
runs/[RUN_FOLDER]/final

- Para Visualizar Layout:

librelane --last-run --flow OpenInKlayout config.json

Comprendiendo STA: Static Timing Analysis

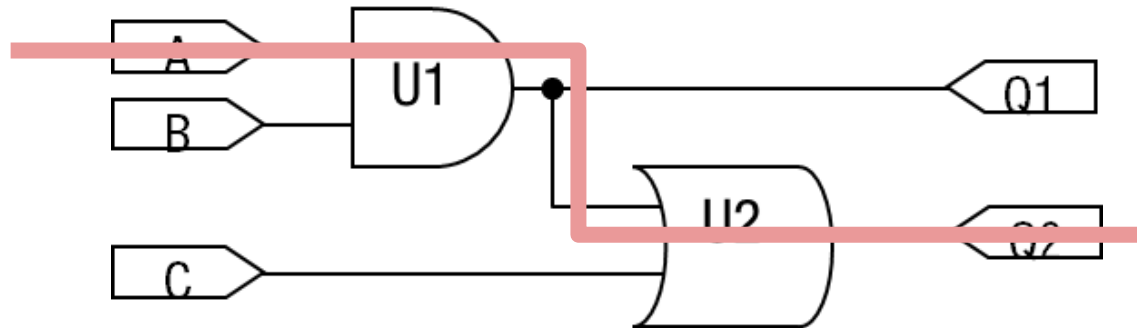


$$t_{\text{AND}} = 3\text{ns}$$

$$t_{\text{OR}} = 5\text{ns}$$

$$t_{\text{ruta-crítica}} = ?$$

Comprendiendo STA: Static Timing Analysis



$$t_{\text{AND}} = 3\text{ns}$$

$$t_{\text{OR}} = 5\text{ns}$$

$$t_{\text{ruta-crítica}} = 8$$

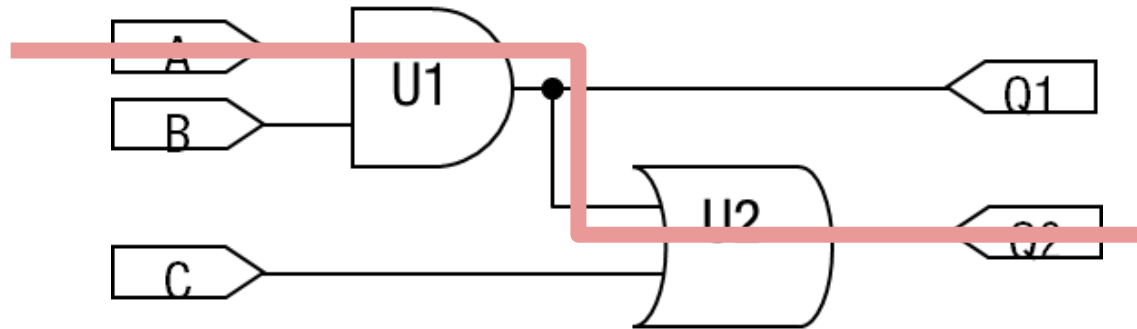
Target Time (ns)	Slack = $t_{\text{target-time}} - t_{\text{ruta-crítica}}$
9	
8	
7	

¿Qué significa slack positivo?

¿Qué significa slack cero?

¿Qué significa slack negativo?

Comprendiendo STA: Static Timing Analysis



$$t_{\text{AND}} = 3\text{ns}$$

$$t_{\text{OR}} = 5\text{ns}$$

$$t_{\text{ruta-crítica}} = 8$$

Target Time (ns)	Slack = $t_{\text{target-time}} - t_{\text{ruta-crítica}}$
9	1
8	0
7	-1

¿Qué significa slack positivo? (Cumple)

¿Qué significa slack cero? (Límite)

¿Qué significa slack negativo? (No cumple)

Comprendiendo STA: Static Timing Analysis

Reportes STA: Pre, Mid, Post (Archivo **summary.rpt**)

12-openroad-staprepnr

Cálculo slack para Placement

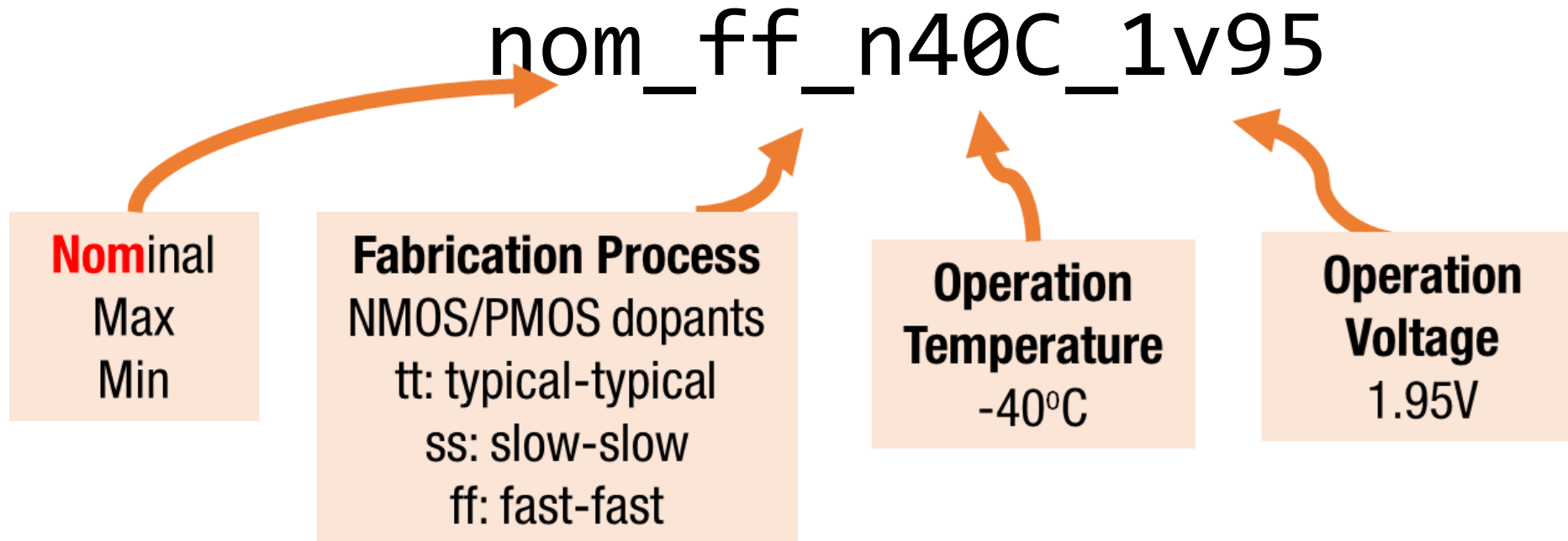
42-openroad-stamidpnr-3

Verificación Setup & Hold

54-openroad-stapostpnr

IPVT corners

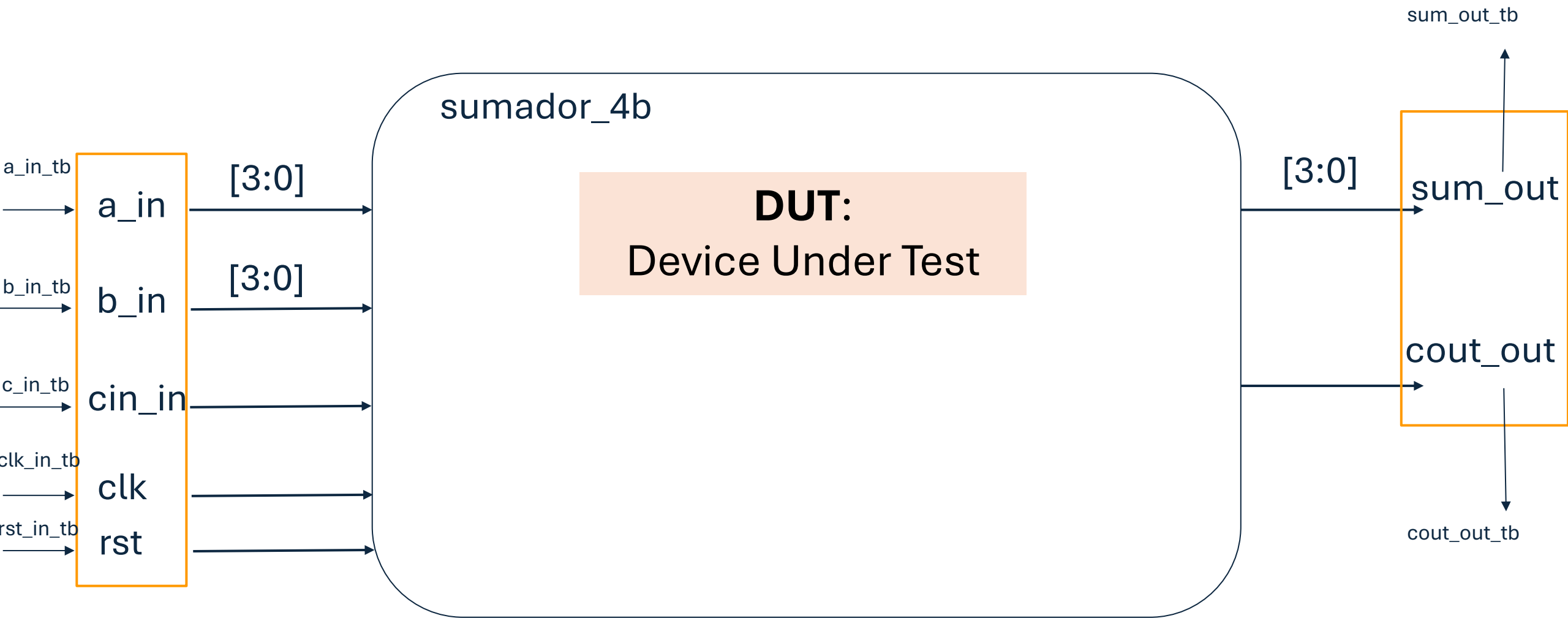
IPVT: Interconnect - Process - Voltage – Temperature



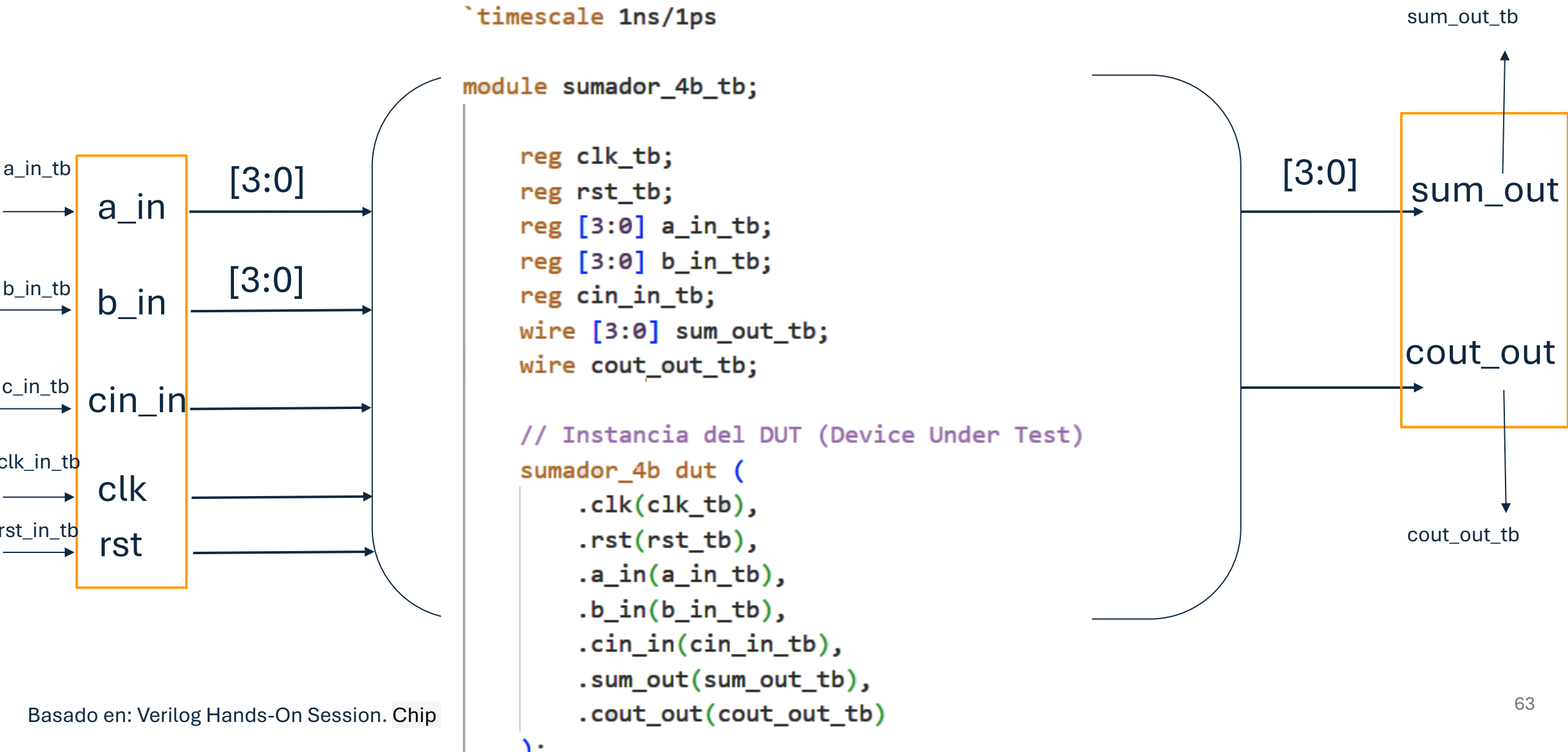
Behavioral Simulation

Simulación en la fase de diseño del chip

Ejemplo Sumador - Testbench



Ejemplo Sumador - Testbench



Ejemplo Sumador

Síntesis:

```
iverilog -o sumador.vvp sumador_4b.v ../test/sumador_4b_tb.v
```

Simulación comportamental:

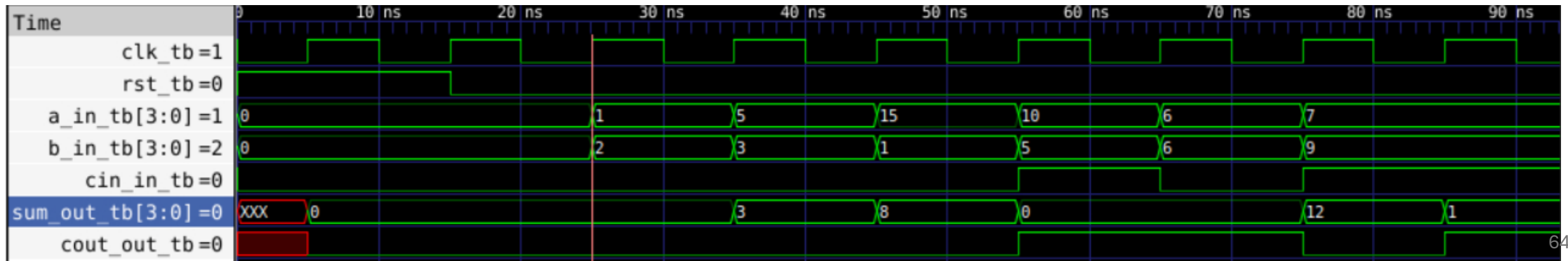
```
vvp sumador.vvp
```

Alternativamente usando el
Makefile del repositorio

```
make test
```

Visualización de señales

```
gtkwave
```



Gate Level Simulation (GLS)

Simulación después de sintetizar el chip

GLS: Gate level simulation

- Al final del proceso se crean los archivos
 - **final/pnl/xxx.pnl.v** Síntesis del circuito en celdas estándares + power
 - **final/nl/xxx.nl.v** Síntesis del circuito en celdas estándares sin power
- Se puede simular con el Testbench de la fase de diseño

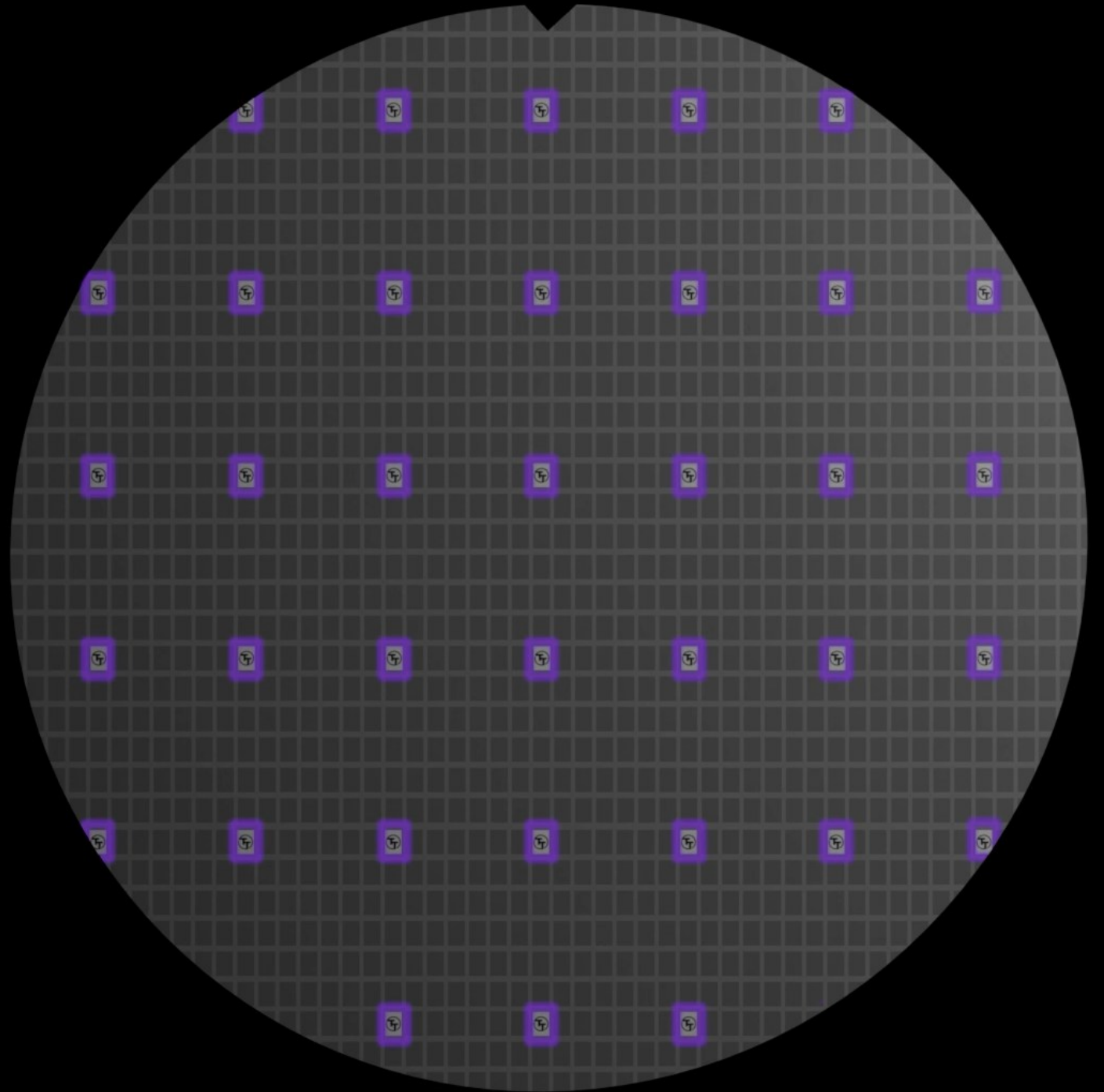
Usar el Makefile del repositorio:

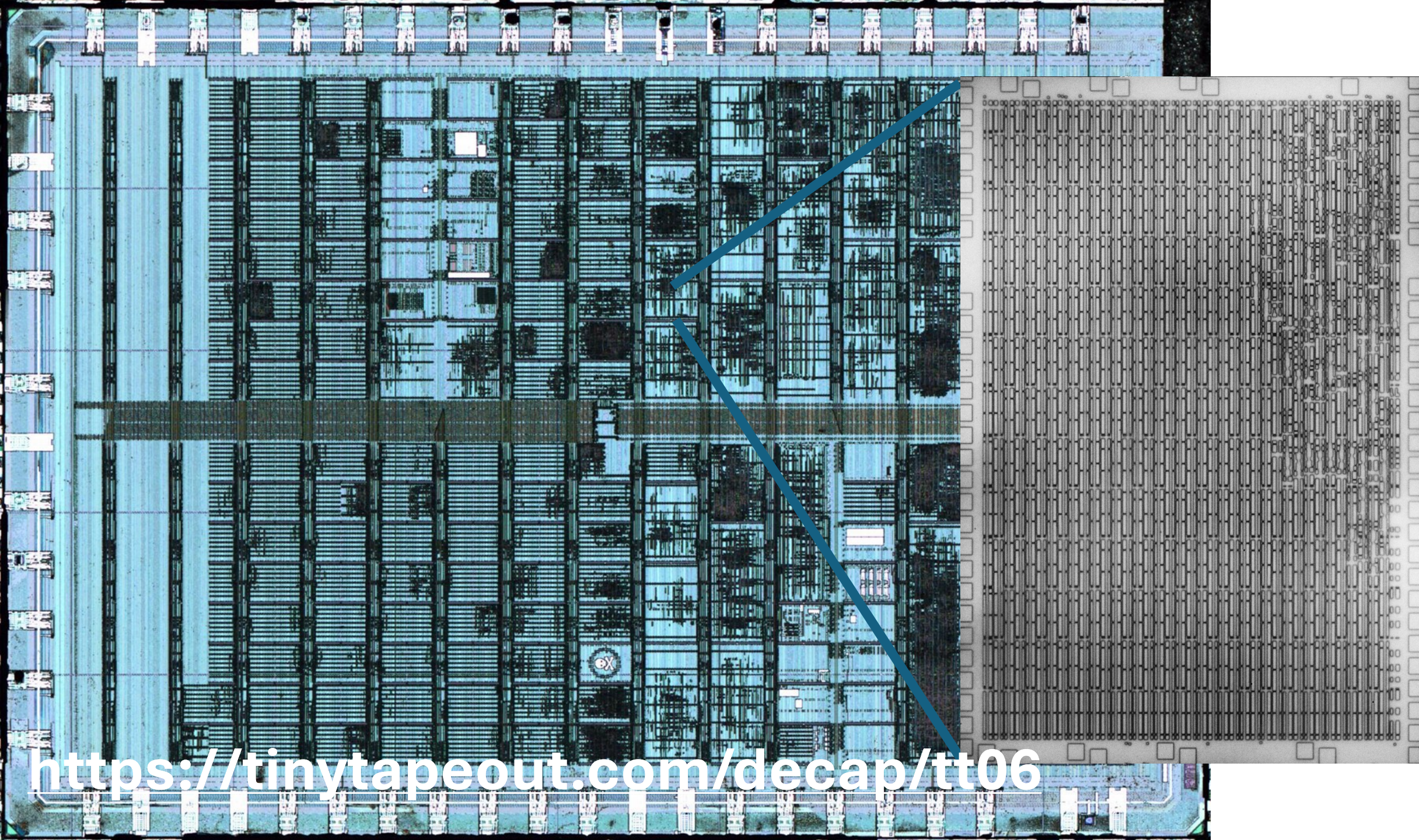
make gls

TinyTapeout

MPW

- Máscaras ~200.000 dólares.
- Oblea multiproyecto abarata costos.
- 1/40 de los chips son para nosotros.

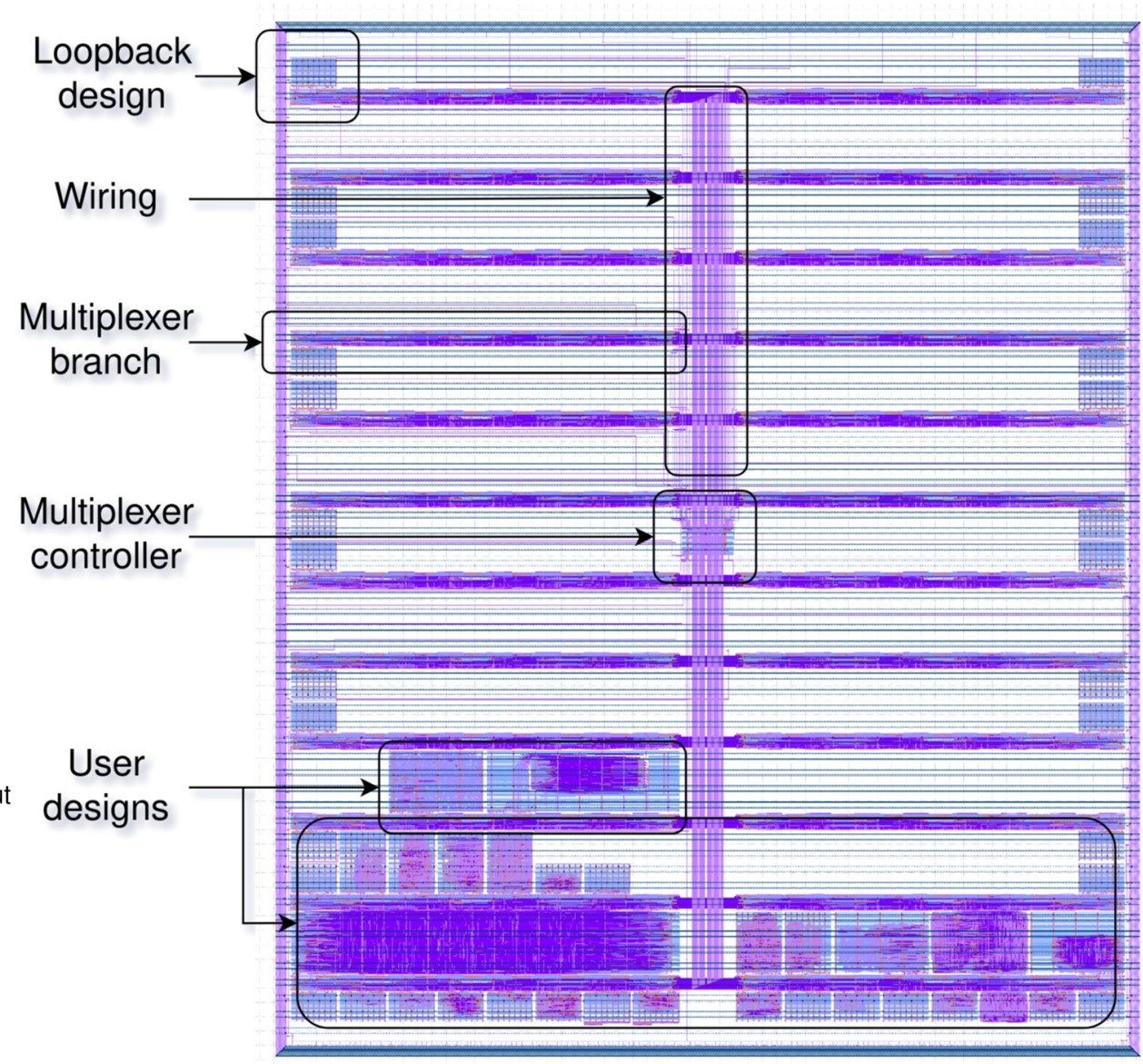
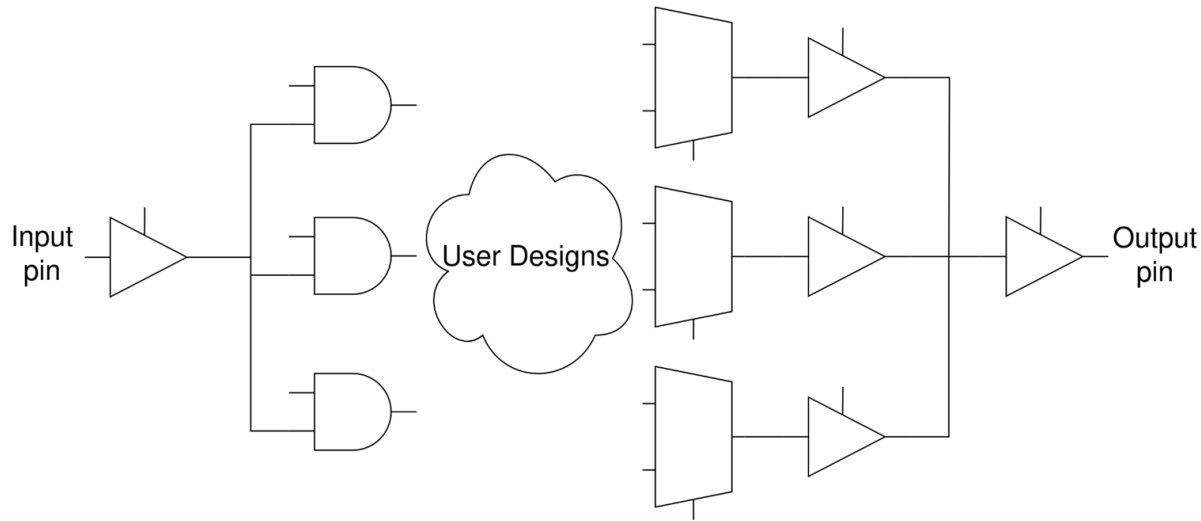




<https://tinytapeout.com/decap/tt06>

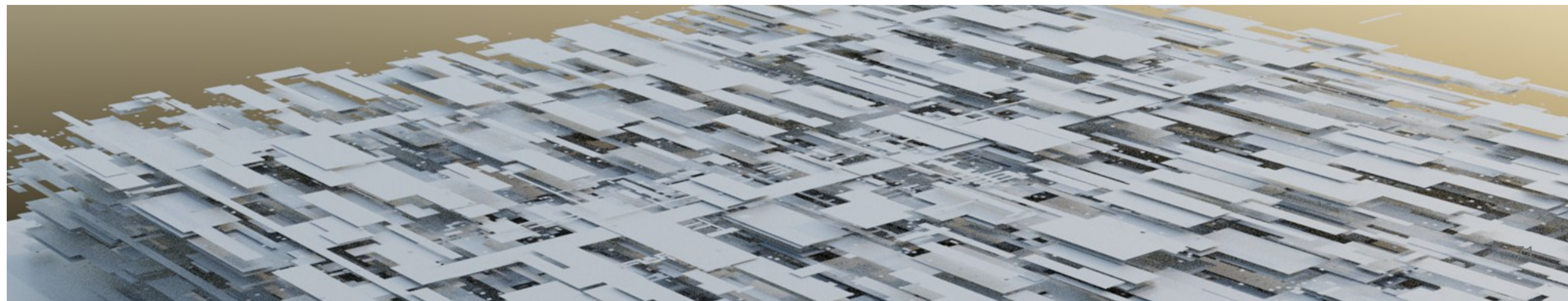
Chip multiproyecto

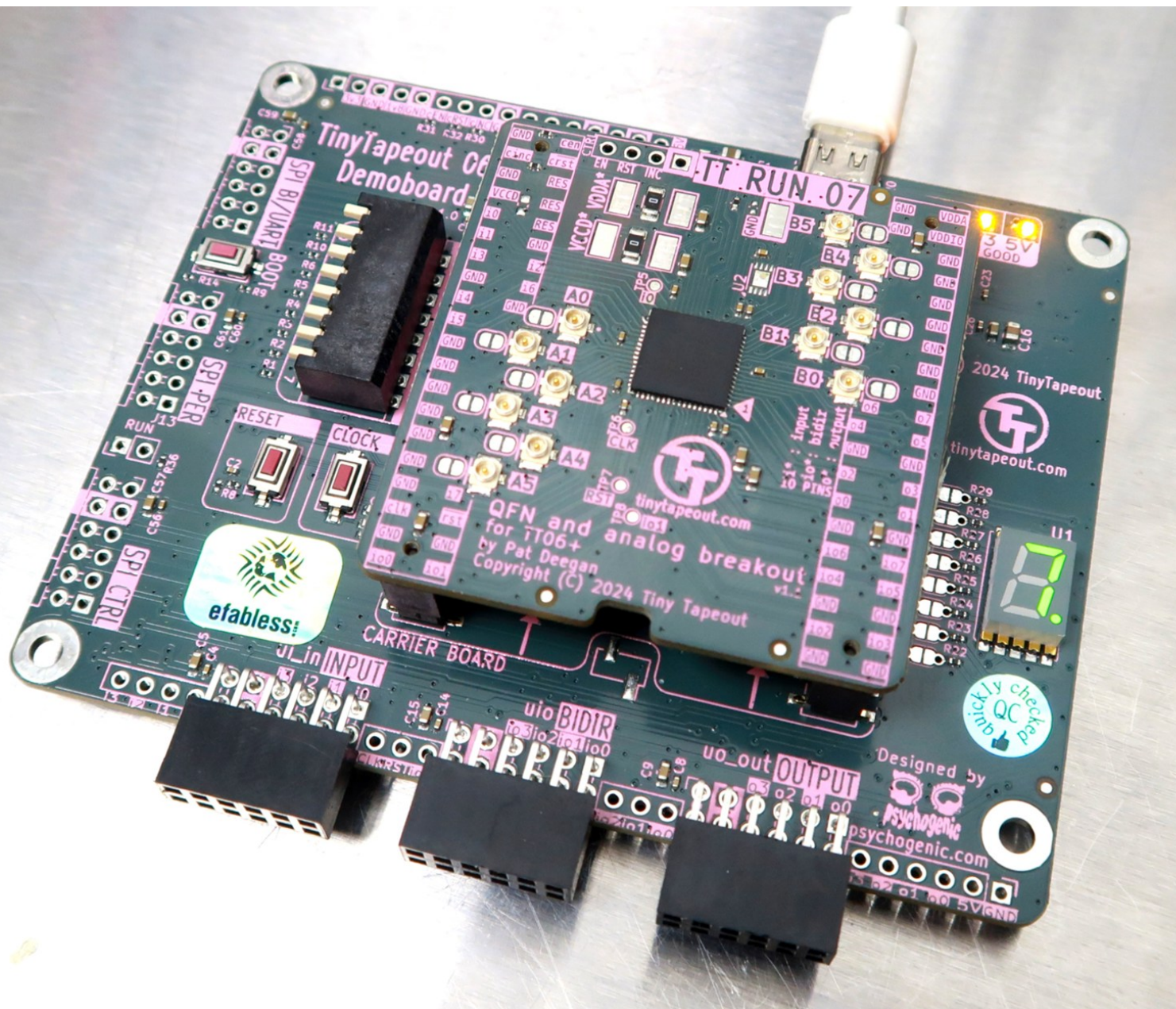
- IO de alta velocidad ~50MHz
- Soporta 512 diseños
- Soporta tiles múltiples
- [\[Paper\]](#)



Limitaciones / Calificaciones máximas

- Diseño digital más grande $8 \times 4 = 1400 \times 500 \text{ um}$ = aproximadamente 50k celdas estándar
- Diseño analógico más grande $3 \times 2 = 600 \times 250 \text{ um}$
- 24 IOs (8 ins, 8 outs, 8 bidirectional), clock & reset, 6 analog pins
- Digital IO bandwidth is $\sim 50\text{MHz}$, round trip latency $\sim 20\text{ns}$
- Metal 5 no permitido
- Página principal de especificaciones: <https://tinytapeout.com/specs>





Configure your Tiny Tapeout Board

For Tiny Tapeout boards with TT04 or a later chip.

Shuttle: [tt06](#)

Firmware: **2.0.4**

[DISCONNECT](#)[CONFIG](#)[INTERACT](#)[PINOUT](#)[REPL](#)

Project

TinyTapeout 06 Factory Test

Index

1

[SELECT](#)

Clock speed (Hz)

10

Preset

[SET](#)

Repo: <https://github.com/TinyTapeout/tt06-factory-test>

[REPORT RESULTS](#)

```
> read_rom()
>>>
raw REPL; CTRL-B to exit
> dump_state()
>OK
sys.version=MicroPython v1.24.0 on 2024-10-25
tt.sdk_version=release_v2.0.4
commit=4f8caad5
repo=TinyTapeout/tinytapeout-06
shuttle=tt06
tt.uo_out=227
tt.design=1
```


Consideraciones para usar con Verilog

- Clonar el repositorio según convenga:
 - Para IHP: <https://github.com/TinyTapeout/ttihp-verilog-template>
 - Para SKY: <https://github.com/TinyTapeout/ttsky-verilog-template>
- Modificar: **info.yaml** & **docs/info.md**
- Incluir archivos Verilog + **Verilog top con formato especial**
- Tamaño del mínimo tile: 160 x 100 um

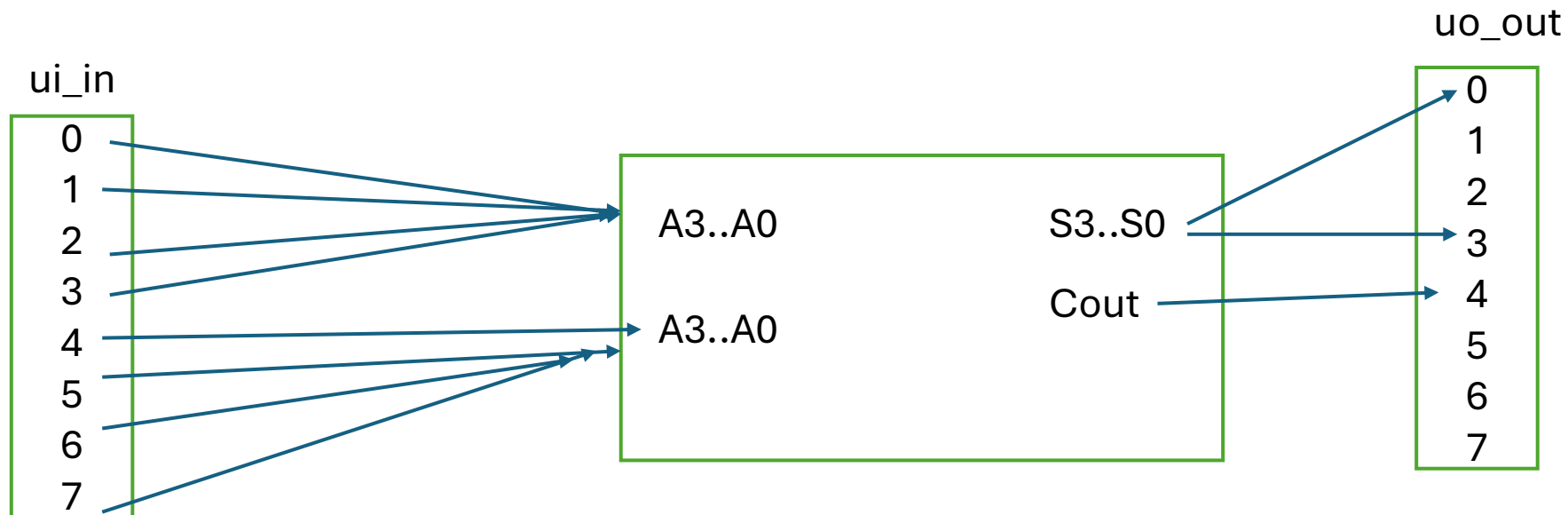
Consideraciones para usar con Verilog

- Modulo top debe llamarse `tt_um_xxx` :

```
module tt_um_xxx (  
    input  wire [7:0] ui_in,      // Dedicated inputs  
    output wire [7:0] uo_out,     // Dedicated outputs  
    input  wire [7:0] uio_in,     // IOs: Input path  
    output wire [7:0] uio_out,    // IOs: Output path  
    output wire [7:0] uio_oe,     // IOs: Enable path (active high: 0=input, 1=output)  
    input  wire      ena,         // will go high when the design is enabled  
    input  wire      clk,        // clock  
    input  wire      rst_n       // reset_n - low to reset  
);
```

Ejemplo Sumador - Entidad top

```
module tt_um_sumador4b (  
    input  wire [7:0] ui_in,      // Dedicated inputs  
    output wire [7:0] uo_out,     // Dedicated outputs  
    input  wire [7:0] uio_in,     // IOs: Input path  
    output wire [7:0] uio_out,    // IOs: Output path  
    output wire [7:0] uio_oe,     // IOs: Enable path (active high: 0=input, 1=output)  
    input  wire      ena,         // will go high when the design is enabled  
    input  wire      clk,        // clock  
    input  wire      rst_n       // reset_n - low to reset  
);
```



Consideraciones para usar con Verilog

- Los pines no usados de entrada se deben conectar a alguna compuerta (eliminar warnings), p.e.:

```
wire _unused = &{ena, uio_in, 1'b0};
```

- Los pines no usados de salida se deben poner a algún nivel (Evitar errores síntesis con Yosys):

```
assign uio_out = 8'bz;
```

```
assign uio_oe = 8'b0;
```

- Debe definir un archivo de pruebas (Carpeta test)
 - TinyTapeout usa cocotb, por lo que debe definir las pruebas en test.py

Paso a paso para sintetizar tu chip en TinyTapeout

- Usar la plantilla

<https://github.com/TinyTapeout/ttsky-verilog-template>

y configurar el repositorio como se hizo con Wokwi (No modificar ningún archivo por ahora).

- Clone su repositorio:

```
git clone https://github.com/[SuUsuario]/[SuProyecto]
```

- Copie **sumador_4b.v** y **tt_um_sumador_4b.v** a la carpeta **src** y borre **project.v**
- **Nota: No modifique config.json!**
- Modifique el archivo **info.yaml** & **docs/info.md**

Crea un archivo de pruebas en cocotb

- Modifica algunas líneas de `test/test.py` para que luzca como:

```
dut._log.info("Test project behavior")  
dut.ui_in.value = 0x21  
await ClockCycles(dut.clk, 3)  
assert dut.uo_out.value == 3
```
- Modifique `test/tb.v`: donde aparezca `tt_um_example` cambiarlo por `tt_um_sumador_4b`
- Modifique la siguiente línea del archivo `Makefile`:

```
PROJECT_SOURCES = sumador_4b.v tt_um_sumador_4b.v
```

Se recomienda probar la simulación antes de enviar los archivos a síntesis escribiendo, dentro de la carpeta test:

make -B

Actualización del repositorio y síntesis del chip

- Suba sus archivos locales al repositorio Github:

```
git add .  
git commit -m "actualizacion"  
git push
```

Debe activar un token en su cuenta de Github y usarlo cuando le pida la contraseña de su Cuenta.

Su repositorio completo debe lucir como este
https://github.com/jimarinh/michip_taller

Gracias

Jorge Iván Marín Hurtado, PhD
jorgemarin@uniquindio.edu.co